



Department of Computer Science

Final Year Project Proposal

Project Name:

Smart Academic Advisory & Student Success Platform (SAASSP)

Supervisor:

Mrs IN Ezeji

Submitted by:

Student Name	Student Number
Nhlanhlehle Ngcobo	240035192
Celimpilo Dlamini	202020444
Zinhle Mhlongo	202285904
Mphilisi Hlandze	230013633
Nokwanda Sibiya	230015229
Thandiwe Mvelase	240111090

Date: 27/07/26

Table of Contents

1. Introduction and Background	4
1.1 Introduction.....	4
1.2 Background	4
2. Problem Statement	5
3. Project Aim and Objectives	6
3.1 Aim	6
3.2 Objectives	6
4. Methodology	6
4.1 Requirements Identification.....	7
4.2 System Design.....	7
4.3 Student Mark Input and Assessment Structure	7
4.4 Continuous, Weighted DP Progress Calculation.....	8
4.5 AI-Assisted Progress Feedback.....	9
4.6 Testing and Evaluation.....	10
4.7 Deployment.....	10
5. Equipment, Tools and Technologies Required	10
5.1 Hardware Requirements.....	10
5.2 Development Technologies.....	10
5.3 Database Technologies	11
5.4 Testing and Collaboration Tools.....	11
5.5 Documentation Tools.....	12

6. Project Scope.....	12
6.1 Minimum Viable Product.....	12
6.2 Optional Enhancement.....	12
6.3 Out of Scope.....	13
7. Intended Users	13
7.1 Students.....	13
7.2 Academic Staff Responsible for Advising.....	14
7.3 System Administrators.....	14
7.4 University.....	14
8. Assumptions, Constraints and Limitations	14
8.1 Assumptions.....	14
8.2 Constraints	15
8.3 Limitations	15
9. Project Risks and Mitigation	15
10. Project Work Plan	16
10.1 Expertise of Team Members.....	16
10.2 Agile Work Plan.....	17
11. References	18
12. Appendices	18
Appendix A: Meeting Minutes	18

1. Introduction and Background

1.1 Introduction

Academic advising supports university students in tracking their assessment performance and understanding their progress towards a module's Duly Performed (DP) requirement. Student information systems and learning management systems already provide access to marks, registrations and academic records, but this information is often spread across separate sources and screens. A student may be able to see those individual test or assignment marks without being able to see, in one place, how those marks add up towards passing a module.

The Smart Academic Advisory & Student Success Platform (SAASSP) is proposed as a web-based academic decision support prototype that addresses this gap. For each module a student selects, the student configures the module's own assessment structure across up to five categories: Tests, Assignments, Quizzes, Practicals and Tutorials, specifying per category, its percentage contribution towards the module's Duly Performed (DP) mark, the total number of assessments expected in that category, and how many of those assessments will be recorded towards DP. As the student uploads the mark obtained for each assessment, SAASSP recalculates a continuous, weighted DP progress percentage, counting only the best recorded marks in each category where fewer assessments count than are written, and generates AI-assisted feedback that responds to the student's current progress level and to the recent trend of their marks within each assessment category.

SAASSP's core decision-support mechanism is rule-based logic for DP progress calculation. The platform is intended to complement lecturers, programme coordinators and other academic staff responsible for advising. It does not replace their professional judgement, and it does not make official DP or examination-admission decisions.

1.2 Background

Student Information Systems commonly display registration information and final captured results, but displaying a mark is different from explaining what that mark means for a student's progress, or whether that progress is improving. A student who has written some, but not all, of the tests, quizzes, assignments, practicals or tutorials that make up a module's Duly Performed (DP) requirement may find it difficult to work out their current standing, particularly where a category only records the best of several attempts towards DP and a later, better mark can displace an earlier one. Existing systems also rarely respond to a newly captured mark with feedback that reflects whether a student's performance in that assessment category is improving, declining or fluctuating, leaving students without timely, personalised guidance after each result becomes available.

Academic staff responsible for advising currently need to combine information manually from academic records and module assessment breakdowns before they can advise a student accurately. SAASSP addresses

this decision-support gap within a controlled prototype by allowing students to input their own marks per assessment component, converting configured module and programme rules into explicit system logic, and presenting the resulting DP progress and AI-assisted feedback through student and academic-staff dashboards.

The prototype will initially represent one selected academic programme rather than the entire university, using the university's approved programme and module data programme codes and names, and the module codes and names linked to each programme to populate programme and module selection at sign-up. Once a student has selected their programme and modules, they configure each selected module's own assessment structure (category weightings, number of assessments per category and number of marks recorded towards DP), within validation rules enforced by the system.

2. Problem Statement

The central problem addressed by this project is that students and academic staff responsible for advising lack an integrated tool that translates individual assessment marks into understandable, actionable guidance. Students can typically see marks for individual tests, assignments, quizzes, practicals and tutorials once released, but existing systems do not consistently allow students to record their own marks as they receive them, or to see, mark by mark, how close they are to a module's Duly Performed requirement, particularly where a category only counts the best of several recorded attempts. Existing systems also do not provide feedback that responds to a student's evolving pattern of performance encouraging a student who is improving, flagging concern for a student whose marks are declining, or acknowledging a student who has recovered from a poor early result leaving students without timely, personalised guidance after each assessment result becomes available.

Academic staff such as lecturers, programme coordinators or other designated advising staff must currently combine the same information manually from student records and module assessment structures, making advising slower and more dependent on manual interpretation. The proposed prototype addresses this by letting students configure each selected module's assessment categories: Tests, Assignments, Quizzes, Practical and Tutorials, with their own DP weightings, assessment counts and recorded mark counts; by giving students a structured way to upload a mark for each assessment as it becomes available; by calculating a continuous, weighted DP progress percentage from those entries, counting only the best recorded marks per category; and by generating AI-assisted feedback after every upload that responds to the student's current DP standing and to the recent trend of their marks, all within one web-based platform.

The prototype will not perform official registration, replace academic staff or make official academic decisions. Its purpose is to demonstrate the technical feasibility of integrating mark capture, DP progress calculation and AI-assisted feedback in a controlled environment.

3. Project Aim and Objectives

3.1 Aim

To design, develop and evaluate a web-based academic decision-support platform that allows students to record their own assessment marks and that uses those marks to calculate continuous, weighted Duly Performed (DP) progress and generate AI-assisted feedback on that progress.

3.2 Objectives

1. To investigate existing academic advising systems, student success platforms and current mark-capture and DP-monitoring processes, to identify their limitations and determine the functional and non-functional requirements of the proposed platform.
2. To design the architecture, database, user interfaces and decision-support components of SAASSP, including a configurable assessment-structure interface (category weightings, assessment counts and recorded-mark counts) and a mark-upload interface, for each module a student selects.
3. To develop a web-based platform that lets students select their programme and modules from approved programme data, configure each module's assessment categories, and upload a mark per assessment; the platform calculates a continuous, weighted DP progress percentage from those marks, counting only the best-recorded marks per category.
4. To develop an AI-assisted feedback mechanism, using a large language model, that generates a response to each mark a student uploads, based on the resulting DP progress percentage, its position within defined safety/risk threshold bands, and the recent trend of the student's marks within that assessment category.
5. To evaluate the integrated platform in terms of calculation correctness, functional correctness, usability, security and compliance with the specified requirements.

4. Methodology

Agile software development will be used as the project's iterative methodology. Rather than treating development as a sequence of fixed phases completed once, the team will work in short sprints, each involving planning, development, testing, review and refinement, so that integration and testing occur throughout development rather than only at the end (Pressman & Maxim, 2020; Schwaber & Sutherland, 2020).

4.1 Requirements Identification

- Review relevant literature on academic advising, mark-based DP monitoring and student decision-support systems.

- Analyse approved programme documents to identify modules, assessment structures (tests, assignments, quizzes, practicals, tutorials), weightings and DP thresholds.
- Review the existing academic advising and mark-recording process relevant to the selected programme.
- Consult the supervisor and, where institutional permission and ethical approval are available, intended users such as students and academic staff responsible for advising.
- Where participant-based requirements gathering is not approved or feasible, rely on literature, approved programme documents, existing advising processes and supervisor-guided consultation.

4.2 System Design

- Define a shared system architecture and database design before separate component development, including an assessment-and-marks schema that supports a configurable number of categories, assessments and recorded-mark counts per module.
- Design user interfaces and role-based dashboards, including a student mark-entry form and a DP progress dashboard.
- Convert programme structures, assessment weightings and DP thresholds into explicit, configurable system rules.
- Develop UML diagrams (architecture, ERD, use-case and activity diagrams) and other system models required for the project.

4.3 Student Mark Input and Assessment Structure

Each module a student selects has its own configurable assessment structure, made up of up to five categories: Tests, Assignments, Quizzes, Practical and Tutorials. For each category, the student configures three values: its percentage contribution towards the module's DP mark, the total number of assessments that category will have during the semester, and how many of those assessments will be recorded towards DP. A category configured at 0% contribution is excluded from the DP calculation entirely. Students will be able to upload the mark obtained for each assessment through a structured upload form, selecting the module and category and entering the percentage achieved. The system will validate that entered marks fall within a valid percentage range, store each entry as a student-provided (provisional) record, and allow correction of an entry before it is used in progress calculations. In the initial prototype, this assessment-structure configuration is entered by the student for each module they select; enabling lecturers to configure it instead, for the modules they teach, is planned as a future extension (Section 6.2).

Category weightings entered by the student are validated in real time against a running total, so the student can see immediately whether their configured categories sum to 100% before the configuration is accepted. The table below shows an illustrative configuration a student might enter for one module, where Tutorials has been set to 0% and is therefore excluded from the DP calculation.

Assessment category	Contribution to DP	Total assessments	Recorded towards DP
Tests	50%	3	2
Quizzes	20%	3	2
Assignments	20%	2	2
Practicals	10%	2	1
Tutorials	0% (excluded from DP calculation)	2	0

- A dedicated home screen listing a student's selected modules, and a per-module upload screen showing configured categories, each with its DP weighting, total assessments and recorded count.
- Input validation to prevent marks outside the valid percentage range, and real-time validation that a module's category weightings sum to 100% before the configuration is accepted.
- A running summary, per category, showing how many assessments have been uploaded, which of those are currently recorded towards DP, and which have been displaced by a later, better mark.
- Clear labelling that student-entered marks are provisional until confirmed against official academic records, where available.

4.4 Continuous, Weighted DP Progress Calculation

For each category, SAASSP maintains two records: the full, ordered history of every mark the student has uploaded in that category, and a trimmed pool containing only the best-recorded marks, capped at the number the student configured as "recorded" for that category. When a new mark is uploaded and the category's recorded cap is already reached, the system compares it against the lowest mark currently in the trimmed pool;

if the new mark is higher, it replaces the lowest, which is then excluded from further DP calculations while remaining visible in the category's full history. A module's overall DP progress percentage is calculated as the sum of DP points earned across every recorded mark in every non-zero-weighted category, divided by the sum of DP points attempted (the weighting share represented by each recorded mark so far), expressed as a percentage. This is the same formula whether the module is mid-semester or complete, so the progress percentage shown at any point is directly comparable to the module's final DP outcome. The resulting percentage is mapped to one of six threshold bands 75-100% very safe/excellent, 70-74% safe, 60-69% generally safe, 50-59% pass but some risk, 40-49% danger/at risk of failing, and 0-39% critical/failing, which is shown to the student alongside the figure. Where attendance or other compulsory-component rules are documented for the selected programme, these will also be represented. Every DP figure will be displayed together with the calculation and the rule it is based on and will never be labelled as an official examination-admission decision.

4.5 AI-Assisted Progress Feedback

After every mark a student uploads including the very first mark ever entered for a module, and marks that are later displaced from a category's recorded pool by a better result SAASSP generates a short piece of feedback using a large language model (Google's Gemini). The band a student's progress falls into, and the trend of their marks within the affected category, are determined deterministically by system logic and are never decided by the model itself; the model's role is limited to producing the wording of the feedback message, based on structured data it is given, and it does not operate as an open-ended chatbot or retain conversational memory between uploads.

Per category trend is classified from the category's full mark history (at least three uploads are required before a trend is classified): a consistent upward or downward run, tolerant of a single reversal, is classified as increasing or dropping respectively; two or more reversals are classified as fluctuating; and a category with only one or two marks so far has no trend yet. The feedback prompt is given the student's current overall DP progress percentage and threshold band, the category and mark just uploaded, that category's trend classification, and the category's full upload history so that the model can reference a mark that no longer counts towards DP where it is narratively relevant, for example, acknowledging a student who performed poorly early on but has since recovered. Feedback is intended to encourage a student who is improving or performing well, build confidence in a student who started poorly but is now improving, and offer concrete, constructive suggestions to a student whose progress or trend indicates risk, without providing precise dosage-style academic instructions or replacing the judgement of academic staff.

4.6 Testing and Evaluation

- Unit testing of DP-calculation, threshold-band classification and per category trend classification functions using Pytest.
- Integration testing of interactions between the database and web application.
- Functional testing against the specified requirements, including mark-entry validation.
- Security testing of authentication, authorisation, input validation and role restrictions.
- Scenario-based correctness testing using predefined student profiles with known expected DP outcomes and outstanding-mark calculations.
- Usability testing or heuristic evaluation, and user-acceptance testing, depending on available participants and approvals.

4.7 Deployment

The completed system will be deployed to a controlled demonstration environment for project evaluation, using representative sample data rather than real student records. Deployment will not constitute implementation within the university's operational systems, and no operational integration or official decision making is proposed.

5. Equipment, Tools and Technologies Required

5.1 Hardware Requirements

Hardware	Purpose
Laptop or desktop computer	System development, testing and demonstration
Stable internet connection	Development resources, collaboration and controlled service access
External or cloud storage	Backup and protection of project files

5.2 Development Technologies

Technology	Purpose and justification
Visual Studio Code	Source code development and project workspace
Python	Backend development and mark/DP calculations
Flask	Application backend; integrates the Python web application with the database and the Gemini API
HTML5	Web page structure, including mark-entry forms
CSS3	User interface styling
Bootstrap 5	Responsive interface design
JavaScript	Client-side interactivity and mark-entry validation
Chart.js	Interactive DP progress dashboard visualisations
Google Gemini API	Generates one-shot AI-assisted feedback text from structured DP progress, threshold band and trend data

5.3 Database Technologies

Technology	Purpose
MySQL	Relational storage for users, programme information, modules, assessment components, student-entered marks and DP calculations
XAMPP	Local development environment providing the MySQL database

Flask will provide the application backend. XAMPP will provide the local MySQL database environment only, and will not be treated as the web-application server.

5.4 Testing and Collaboration Tools

Tool	Purpose
Pytest	Python unit and component testing, including DP calculation tests
Postman	API testing where applicable
Git	Version control
GitHub	Repository hosting, branches, issues, collaboration and code reviews

5.5 Documentation Tools

Tool	Purpose
Microsoft Word	Project documentation
Draw.io / Lucidchart	UML diagrams and system design

Software and library versions used during implementation will be recorded in the final project documentation to support reproducibility.

6. Project Scope

6.1 Minimum Viable Product

The minimum viable product will initially support one selected academic programme and will include:

- Authentication and role-based access for students, academic staff and administrators.
- Management of programme and module information sourced from approved programme data, and a student-configurable assessment-structure per module (category weightings, assessment counts and recorded-mark counts).

- A student mark-upload interface for capturing marks per assessment, with input validation and real-time category-weighting validation.
- Continuous, weighted DP progress calculation per module, using only the best-recorded marks per category, mapped to defined safety/risk threshold bands.
- AI-assisted feedback, generated after every mark upload, that responds to the student's current DP progress band and to their recent trend within that assessment category.
- Student and academic-staff dashboards.
- Basic reporting and visualisation.

DP progress information will be indicative and dependent on the assessment-structure configuration entered for each module.

6.2 Optional Enhancement

Two enhancements are planned beyond the core student-facing platform, to be attempted only once mark upload, DP progress calculation and AI-assisted feedback have been completed and tested:

- Lecturer-configured assessment structures. Lecturers would log into their own dashboard, select the modules they lecture, and set each module's category weightings, assessment counts and recorded-mark counts on behalf of their students, rather than each student configuring these values individually. In the initial prototype, only students configure their own modules' assessment structure; enabling this for lecturers is intended as a later extension.
- A conversational AI Academic Advisor chatbot, distinct from the one-shot, per-upload progress feedback described in Section 4.5. If implemented, it will provide supplementary academic guidance based on configured information through an ongoing conversation and will not replace academic staff or make official decisions.

6.3 Out of Scope

- Integration with the university's Student Information System, Learning Management System or other operational institutional systems.
- Use of real student academic records or official release of marks; all marks are student-entered for prototype purposes.
- Official module registration or automatic registration.

- Automatic timetable generation.
- Examination admission and graduation determination.
- Automatic academic decisions made without academic-staff oversight.
- Mobile application development.
- Online learning or examination functionality.
- Financial aid, bursary or NSFAS management.
- Real-time SMS or email notification services.
- Claims or evaluation of improvements in retention, academic performance, pass rates or graduation rates.

7. Intended Users

7.1 Students

Students are the primary intended users. They will be able to configure their own modules' assessment categories, upload marks for each assessment as results are released, see an up-to-date weighted DP progress percentage and threshold band per module, and receive AI-assisted feedback after every upload that responds to their current standing and recent trend.

7.2 Academic Staff Responsible for Advising

Academic staff responsible for advising, including lecturers, programme coordinators and other designated staff, are also primary intended users. In the initial prototype, academic staff have a read-only view of a student's mark history and DP progress to support advising sessions, while a module's assessment-structure configuration is entered by the student. It supports, rather than replaces, their professional judgement. As described in Section 6.2, enabling lecturers to configure the assessment structure for the modules they teach, rather than leaving this to individual students, is planned as a future extension.

7.3 System Administrators

System administrators will maintain user accounts, programme structures, module and assessment configurations, and configured academic rules. Where necessary, a programme coordinator may be assigned an academic-advisor role with additional permissions to maintain or verify programme and DP rules.

7.4 University

The university is an indirect beneficiary because the project demonstrates a possible approach to integrated mark capture, DP progress monitoring and AI-assisted feedback. The proposal does not claim that the prototype will improve university retention, marks or pass rates.

8. Assumptions, Constraints and Limitations

8.1 Assumptions

- Approved programme, module and assessment weighting information will be available.
- Selected DP and academic rules can be represented in system logic.
- Students will input their own marks accurately, or synthetic profiles will be used for testing.
- Intended users will have access to a browser and a suitable device.
- Academic staff responsible for advising will retain responsibility for final DP decisions.
- Category weightings, assessment counts and recorded-mark counts are configured by the student per module, and must sum to 100% across a module's non-zero-weighted categories before the configuration is accepted by the system.

8.2 Constraints

- The project has limited time and technical resources.
- AI-assisted feedback depends on access to the Gemini API, which is an external, rate limited service outside the team's control.
- The prototype will not integrate with operational university systems.
- Only one selected programme's rules will be represented.
- Access to representative users may be limited.
- User evaluation may require institutional permission or ethical approval.
- Deployment will occur in a controlled demonstration environment.

8.3 Limitations

- Student entered marks are self reported and provisional and may not match official records until confirmed.

- The wording of AI-generated feedback may vary between similar situations, since only the underlying threshold band and trend classification are deterministic; the feedback is supportive in intent and does not constitute academic advice.
- The prototype will cover only configured programmes and academic scenarios.
- A small usability evaluation cannot represent all university users.
- The project cannot establish improvements in academic performance or retention.

9. Project Risks and Mitigation

Risk	Potential impact	Mitigation
Scope expansion	Too many features may prevent completion.	Prioritise the MVP and treat the chatbot as optional.
Incorrect DP rules	Misleading DP progress information.	Use approved programme documents and scenario-based verification.
Incorrect or inconsistent student mark entry	Misleading DP calculations and outstanding-mark warnings.	Input validation, provisional-data labelling and boundary value calculation tests.
Component integration problems	System components may not work together.	Use a shared architecture, database design, coding standards and continuous integration.
Team coordination problems	Tasks may be duplicated or delayed.	Use GitHub, regular meetings, clear task allocation and sprint reviews.
Loss of project files	Source code or documentation may be lost.	Use Git version control and regular backups.
Security weaknesses	Unauthorised access or incorrect role restrictions.	Apply secure password handling, input validation and role-based testing.

Risk	Potential impact	Mitigation
Limited user evaluation	Participant access or ethical approval may be restricted.	Use heuristic evaluation, supervisor-guided review and structured scenario testing.
External-service dependency	The Gemini API may introduce cost, rate-limit or availability issues affecting feedback generation.	Cache or gracefully degrade feedback generation if the API is temporarily unavailable, and monitor usage against quota.
Inappropriate or inconsistent AI-generated feedback	Feedback could feel discouraging, inconsistent, or fail to reflect the underlying DP data.	Keep threshold-band and trend decisions rule-based, not model-decided; supply only structured data to the model; review generated wording against defined tone guidelines during testing.

10. Project Work Plan

10.1 Expertise of Team Members

The project team consists of six members with complementary skills in systems analysis and design, web development, database design, documentation, research and data preparation. Primary responsibilities are allocated to members, while reviews and integration activities involve the wider team.

Team Member	Role / Expertise	Primary Responsibilities
N. S. Ngcobo (Team Leader)	Systems Analyst & Web Developer	Requirements analysis, system design, database design, web development and project coordination.
C. N. Dlamini	Research & Documentation	Literature review and project documentation.

Team Member	Role / Expertise	Primary Responsibilities
T. Mvelase	Frontend Developer	Mark-entry and dashboard development, and frontend development.
Z. Mhlongo	System Developer	System design and web development.
M. A. Hlandze	Documentation & Data Preparation	Project documentation and test-data preparation.
N. Sibiya	Database & Web Developer	Database design (including marks and DP schema), implementation and web development.

10.2 Agile Work Plan

The work plan follows iterative Agile development. Requirements identification, programme and assessment-rule analysis and MVP definition begin the project. Shared architecture, database design and interface standards are agreed before separate components are developed. Integration, testing, rule validation and defect correction occur throughout the sprints.

Sprint / Period	Main Activities
Sprint 1: August	Requirements identification, literature review, programme and assessment-structure analysis and MVP definition.
Sprint 2: August/September	Shared system architecture, database design (including marks/DP schema), interface standards and authentication/user roles.
Sprint 3: September	Programme, module and synthetic record management; assessment-weighting configuration.

Sprint / Period	Main Activities
Sprint 4: September/October	Student assessment-configuration and mark-upload interface for all categories; continuous, weighted DP progress calculation, threshold bands and AI-assisted feedback.
Sprint 5: October	Refinement of AI-assisted feedback generation (prompt design, threshold-band and trend logic testing), dashboard visualisations, and student and academic-staff dashboards.
Sprint 6: October	Preparation of scenario-based test data with known expected DP outcomes, and accessibility and usability improvements.
Sprint 7: October	System integration, security testing, functional testing and scenario-based DP correctness testing.
Sprint 8: October	Usability evaluation, defect correction, documentation, final validation and controlled demonstration.

The team will use GitHub branches, issues, pull requests/code reviews and regular meetings to coordinate the six-member team. Primary responsibilities will be assigned to individual members, but the mark-entry and DP calculation components will be reviewed by other team members. The final report will document each member's contribution while presenting SAASSP as one integrated system.

11. References

- Laudon, K. C., & Laudon, J. P. (2022). Management information systems: Managing the digital firm (17th ed.). Pearson.
- Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill.
- Schwaber, K., & Sutherland, J. (2020). The Scrum Guide: The definitive guide to Scrum: The rules of the game. <https://scrumguides.org/scrum-guide.html>
- Sommerville, I. (2021). Software engineering (10th ed.). Pearson.

12. Appendices

Appendix A: Meeting Minutes



Project_Meeting_Minutes_Group20.pdf



Department of Computer Science

Software Requirements Specification

Project Title:

SMART ACADEMIC ADVISORY & STUDENT SUCCESS PLATFORM (SAASSP)

Supervisor:

Mrs IN Ezeji

Submitted by:

Student Name	Student Number
Nhlanhlehle Ngcobo	240035192
Celimpilo Dlamini	202020444
Zinhle Mhlongo	202285904
Mphilisi Hlandze	230013633
Nokwanda Sibiya	230015229
Thandiwe Mvelase	240111090

Date: 02/09/2026

Table of Contents

1.0. Introduction

- 1.1. Purpose
- 1.2. Scope of Project
- 1.3. Glossary
- 1.4. References
- 1.5. Overview of Document

2.0. Overall Description

- 2.1 System Environment
- 2.2 Functional Requirements Specification
 - 2.2.1 Student Use Cases
 - 2.2.2 Academic Staff Use Case
 - 2.2.3 System Administrator Use Cases
- 2.3 User Characteristics
- 2.4 Non-Functional Requirements

3.0. Requirements Specification

- 3.1 External Interface Requirements
- 3.2 Functional Requirements
 - 3.2.1 Login (Student)
 - 3.2.2 Register and Select Programme and Modules
 - 3.2.3 Configure Assessment Structure
 - 3.2.4 Upload Assessment Mark

3.2.5 Receive AI-Assisted Feedback

3.2.6 View DP Progress Dashboard

3.2.7 Login (Academic Staff)

3.2.8 View Student Progress

3.2.9 Login (Administrator)

3.2.10 Manage Programme and Module Data

3.2.11 Manage User Accounts

3.3 Detailed Non-Functional Requirements

3.3.1 Logical Structure of the Data

3.3.2 Security

1.0. Introduction

1.1. Purpose

The purpose of this document is to present a detailed description of the Smart Academic Advisory & Student Success Platform (SAASSP). It explains the purpose and features of the system, the interfaces of the system, what the system will do, the constraints under which it must operate, and how the system will react to external stimuli. This document is intended for both the project supervisor and the development team, and is submitted in support of the SAASSP final year project.

1.2. Scope of Project

This software system is a web-based prototype that lets a student record their own assessment marks for each module they select, and that calculates a continuous, weighted Duly Performed (DP) progress percentage from those marks. For each module, a student configures up to five assessment categories — Tests, Assignments, Quizzes, Practicals and Tutorials — specifying each category's percentage contribution towards DP, the total number of assessments in that category, and how many of those assessments are recorded towards DP. Where a category records fewer assessments than are written, only the best-recorded marks count, and a later, better mark displaces the lowest currently recorded mark.

After every mark a student uploads, the system recalculates the module's DP progress percentage, maps it to one of six threshold bands, classifies the recent trend of marks within the affected category, and generates a short piece of AI-assisted feedback using the Gemini API. The threshold band and trend classification are determined by deterministic system logic; the Gemini API is responsible only for the wording of the feedback message.

Academic staff have a read-only view of a student's mark history and DP progress to support advising sessions. A System Administrator maintains the underlying programme and module data, sourced from the university's approved programme information, and user accounts.

This prototype does not perform module recommendations, prerequisite or co-requisite checking, credit accumulation or graduation-progress tracking, or machine-learning-based risk classification. Enabling lecturers to configure a module's assessment structure on behalf of their students, and a separate conversational AI Academic Advisor chatbot, are noted in this document as planned future enhancements outside the current scope.

1.3. Glossary

Term	Definition
Assessment Category	One of five configurable groups of assessments for a module: Tests, Assignments, Quizzes, Practicals or Tutorials.
Category Weighting	The percentage a configured assessment category contributes towards a module's DP progress percentage.
DP (Duly Performed)	The continuous academic standing a student must maintain in a module's assessed coursework.
DP Progress Percentage	The weighted percentage, calculated from recorded marks, showing a student's current standing towards a module's DP requirement.
Gemini API	Google's large language model API, used to generate the wording of AI-assisted feedback messages.
Module	A single university course/subject a student is registered for, identified by a module code and module name.
Programme	A university qualification made up of a defined set of modules, identified by a programme code and programme name.
Recorded Mark	An uploaded assessment mark currently counted towards a module's DP progress percentage, as distinct from a mark displaced from the recorded pool by a later, better result.
SAASSP	Smart Academic Advisory & Student Success Platform; the system described by this document.
SRS	Software Requirements Specification; this document.

Term	Definition
Threshold Band	One of six fixed bands (very safe/excellent, safe, generally safe, pass but some risk, danger/at risk of failing, critical/failing) that a DP progress percentage is mapped to.
Trend Classification	A per-category classification (increasing, dropping or fluctuating) derived from the ordered history of a student's uploaded marks in that category.

1.4. References

- IEEE. IEEE Std 830-1998 IEEE Recommended Practice for Software Requirements Specifications. IEEE Computer Society, 1998.
- Laudon, K. C., & Laudon, J. P. (2022). Management information systems: Managing the digital firm (17th ed.). Pearson.
- Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill.
- Schwaber, K., & Sutherland, J. (2020). The Scrum Guide: The definitive guide to Scrum: The rules of the game. <https://scrumguides.org/scrum-guide.html>
- Sommerville, I. (2021). Software engineering (10th ed.). Pearson.

1.5. Overview of Document

The next chapter, the Overall Description section, gives an overview of the functionality of the product. It describes the informal requirements and establishes a context for the technical requirements specification in the next chapter.

The third chapter, the Requirements Specification section, is written primarily for the developers and describes in technical terms the details of the functionality of the product.

Both sections describe the same software product in its entirety, but are intended for different audiences and thus use different language.

2.0. Overall Description

2.1 System Environment

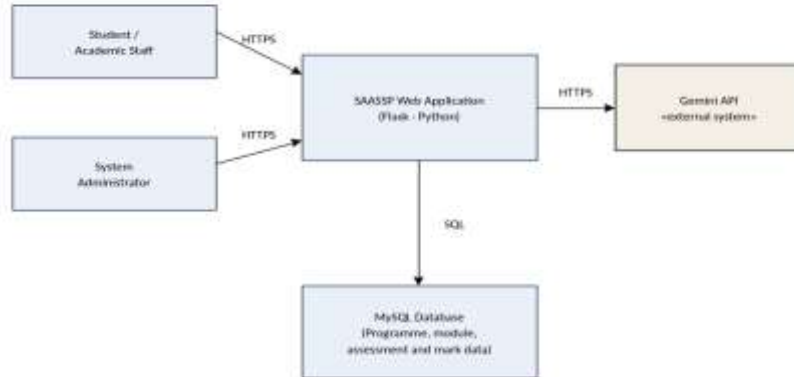


Figure 1 - System Environment

SAASSP has three human actors — Student, Academic Staff and System Administrator — and one cooperating external system, the Gemini API. Students and Academic Staff access the system through a web browser over the Internet. The System Administrator accesses the same web application to maintain programme, module and user-account data. The SAASSP web application (built with Flask) is the single point of access for all actors; it stores programme, module, assessment-configuration and mark data in a MySQL database, and communicates with the Gemini API over HTTPS whenever a piece of AI-assisted feedback needs to be generated.

The Gemini API is external to SAASSP: it receives only the structured data needed to generate feedback wording (the current DP progress percentage, threshold band, the category and mark just uploaded, the trend classification, and the category's upload history) and returns generated text. It plays no part in calculating DP progress, threshold bands or trend classifications, which are performed entirely by SAASSP's own rule-based logic.

2.2 Functional Requirements Specification

This section outlines the use cases for each of the three actors. Login is a precondition for every other use case in this document; because each actor reaches a different home screen and has access to different functionality once signed in, Login is presented separately for each actor below rather than as a single combined use case.

The Student is otherwise the primary actor with the most use cases; Academic Staff and the System Administrator each have a smaller, distinct set of use cases.

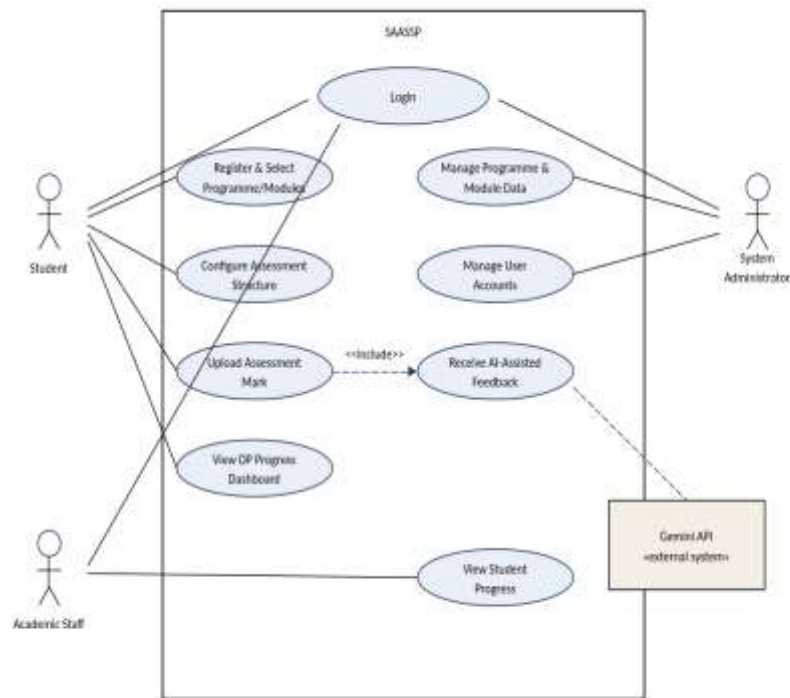
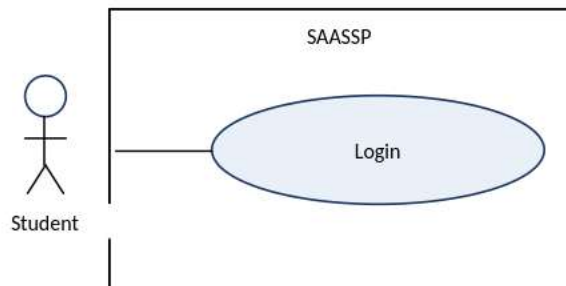


Figure 2 - SAASSP Use Case Diagram

2.2.1 Student Use Cases

Use case: Login (Student)

Diagram:



Brief Description

The Student signs in with an existing account to reach their Dashboard, where they can select modules, configure assessment structures, upload marks and view DP progress.

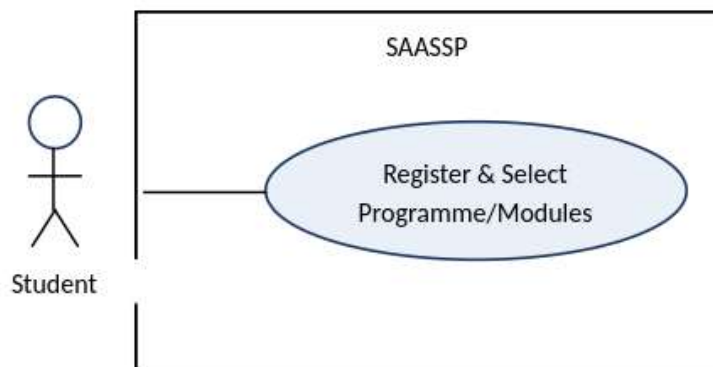
Initial Step-By-Step Description

1. The Student navigates to the sign-in page and enters their email address and password.
2. The system validates the credentials against the stored account record.
3. The system establishes an authenticated session and directs the Student to their Dashboard.

Xref: Section 3.2.1, Login (Student)

Use case: Register and Select Programme and Modules

Diagram:



Brief Description

The Student selects their academic programme and the modules they are registered for, from data sourced from the university's approved programme information.

Initial Step-By-Step Description

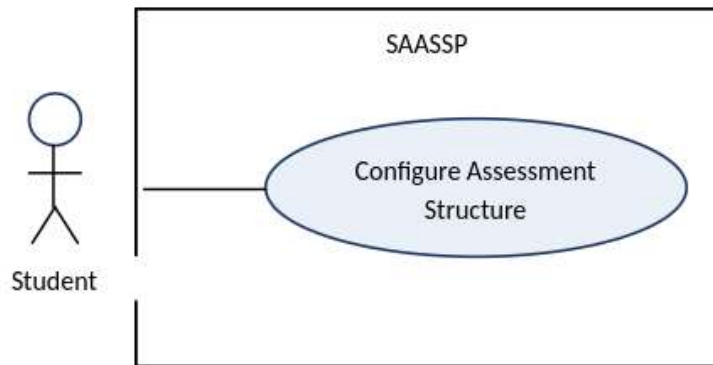
1. The Student, having created an account, chooses to select their programme and modules.
2. The system presents the list of available programmes.
3. The Student selects their programme.
4. The system presents the list of modules linked to the selected programme.
5. The Student selects the modules they are registered for.

6. The system saves the selected programme and modules against the Student's profile.

Xref: Section 3.2.2, Register and Select Programme and Modules

Use case: Configure Assessment Structure

Diagram:



Brief Description

The Student configures the assessment categories for a selected module, setting each category's DP weighting, total assessment count and recorded-mark count.

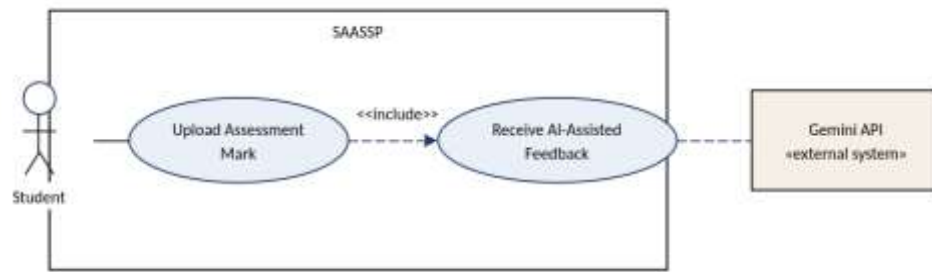
Initial Step-By-Step Description

1. The Student selects a module to configure.
2. The system presents the five assessment categories: Tests, Assignments, Quizzes, Practicals and Tutorials.
3. The Student enters, for each category, its DP weighting, total assessments, and number recorded towards DP.
4. The system validates in real time that the weightings sum to 100%.
5. The Student submits the configuration and the system saves it.

Xref: Section 3.2.3, Configure Assessment Structure

Use case: Upload Assessment Mark

Diagram:



Brief Description

The Student uploads the percentage mark achieved for a specific assessment in a configured category.

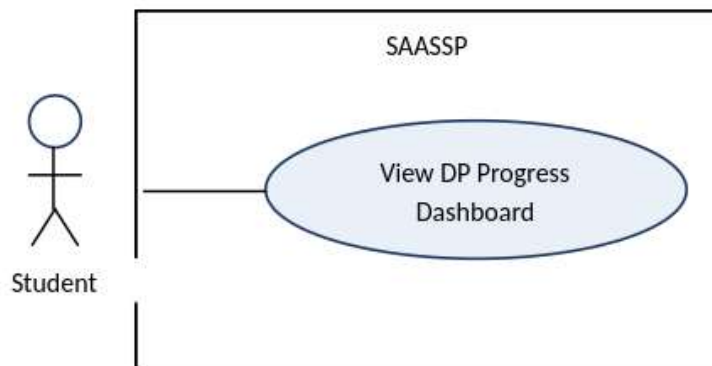
Initial Step-By-Step Description

1. The Student selects the module and assessment category.
2. The Student enters the percentage mark achieved.
3. The system validates the mark and adds it to the category's upload history.
4. The system updates the category's recorded pool, displacing the lowest recorded mark if the cap is already reached and the new mark is higher.
5. The system recalculates the module's DP progress percentage and threshold band.
6. The system invokes the Receive AI-Assisted Feedback use case.

Xref: Section 3.2.4, Upload Assessment Mark

Use case: View DP Progress Dashboard

Diagram:



Brief Description

The Student views a module's current DP progress percentage, threshold band, per-category breakdown, and most recent AI-assisted feedback.

Initial Step-By-Step Description

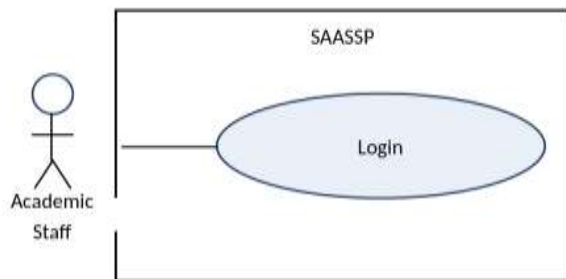
1. The Student selects a module from their home screen.
2. The system displays the module's current DP progress percentage and threshold band.
3. The system displays a per-category breakdown of uploaded and recorded marks.
4. The system displays the most recent AI-assisted feedback message.

Xref: Section 3.2.6, View DP Progress Dashboard

2.2.2 Academic Staff Use Case

Use case: Login (Academic Staff)

Diagram:



Brief Description

The Academic Staff member signs in with an existing account to reach the Students list, from which they review a student's DP progress in read-only form.

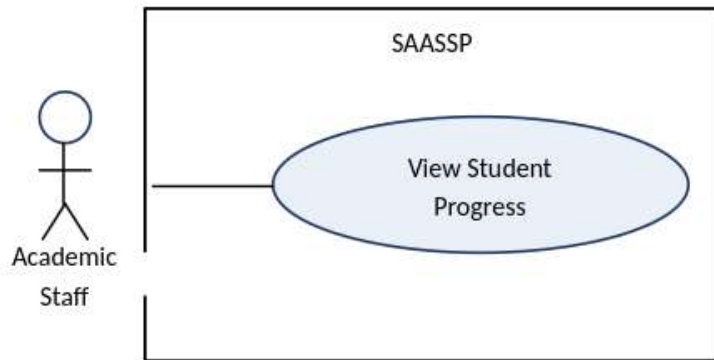
Initial Step-By-Step Description

1. The Academic Staff member navigates to the sign-in page and enters their email address and password.
2. The system validates the credentials against the stored account record.
3. The system establishes an authenticated session and directs the Academic Staff member to their Students list.

Xref: Section 3.2.7, Login (Academic Staff)

Use case: View Student Progress

Diagram:



Brief Description

The Academic Staff member reviews a student's DP progress for a module in read-only form to support an advising session.

Initial Step-By-Step Description

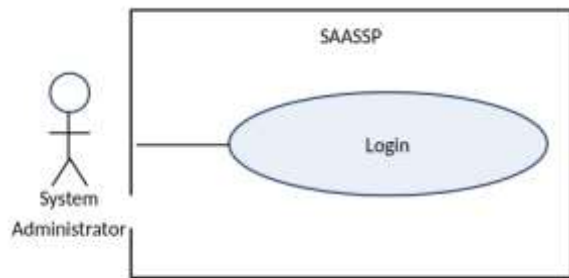
1. The Academic Staff member selects a student from their associated list.
2. The Academic Staff member selects a module.
3. The system displays the module's DP progress percentage, threshold band, per-category breakdown and recent feedback, in read-only form.

Xref: Section 3.2.8, View Student Progress

2.2.3 System Administrator Use Cases

Use case: Login (Administrator)

Diagram:



Brief Description

The System Administrator signs in with an existing account to reach programme/module data management and user-account management.

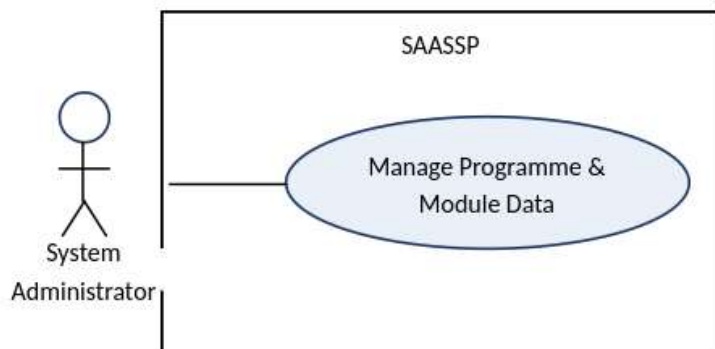
Initial Step-By-Step Description

1. The System Administrator navigates to the sign-in page and enters their email address and password.
2. The system validates the credentials against the stored account record.
3. The system establishes an authenticated session and directs the System Administrator to the programme-data management screen.

Xref: Section 3.2.9, Login (Administrator)

Use case: Manage Programme and Module Data

Diagram:



Brief Description

The System Administrator loads and maintains the approved programme and module data used for Student registration and module selection.

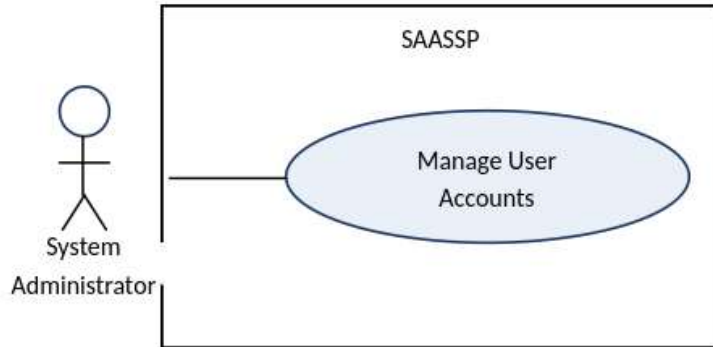
Initial Step-By-Step Description

1. The System Administrator uploads or enters programme data.
2. The System Administrator uploads or enters module data.
3. The System Administrator links modules to programmes.
4. The system validates and stores the data.

Xref: Section 3.2.10, Manage Programme and Module Data

Use case: Manage User Accounts

Diagram:



Brief Description

The System Administrator creates, updates or deactivates user accounts and assigns their role.

Initial Step-By-Step Description

1. The System Administrator selects to view the list of user accounts.
2. The System Administrator selects an account to create, update or deactivate.
3. The system applies the change and updates the account record.

Xref: Section 3.2.11, Manage User Accounts

2.3 User Characteristics

The Student is expected to be Internet literate and familiar with basic web forms; no specialised academic or technical knowledge is assumed beyond knowing their own assessment marks as released to them.

The Academic Staff member is expected to be Internet literate and familiar with reviewing student records during advising sessions.

The System Administrator is expected to be familiar with the university's approved programme documentation and comfortable performing routine data-entry and account-maintenance tasks.

2.4 Non-Functional Requirements

- Performance: a module's DP progress percentage, threshold band and trend classification are recalculated and displayed within a few seconds of a mark being uploaded, independent of the response time of the Gemini API.
- Availability: the core mark-upload and DP progress-calculation functionality remains available even if the Gemini API is temporarily unavailable; only feedback generation is affected.
- Usability: category-weighting validation is performed in real time so a Student can see immediately whether a configuration is valid.
- Reliability: DP progress, threshold-band and trend calculations are deterministic and rule-based, and do not depend on the output of the Gemini API.
- Security: access is role-based (Student, Academic Staff, System Administrator); all communication uses HTTPS.
- Compatibility: the system is accessible through standard, current web browsers.

3.0. Requirements Specification

3.1 External Interface Requirements

The only interface to an external system is the interface to the Gemini API, used to generate the wording of AI-assisted feedback messages. The Receive AI-Assisted Feedback use case (Section 3.2.5) sends a structured HTTPS request containing the module's current DP progress percentage, its threshold band, the assessment category and mark just uploaded, that category's trend classification, and the category's full upload history. The Gemini API returns generated feedback text, which SAASSP stores and displays without further modification.

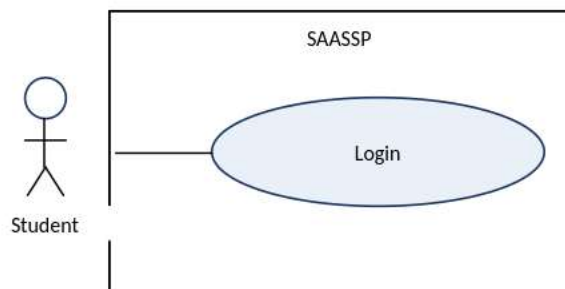
The Gemini API is never sent raw student-identifying information beyond what is required to generate the feedback text, and it never determines the DP progress percentage, threshold band or trend classification, all of which are calculated by SAASSP's own rule-based logic before the request is sent.

3.2 Functional Requirements

The logical structure of the data referenced throughout this section is contained in Section 3.3.1.

3.2.1 Login (Student)

Diagram:



Use Case Name	Login (Student)
XRef	Section 2.2.1, Login (Student)
Trigger	A Student with an existing account chooses to sign in.
Precondition	The Student has a registered account, created via self-service sign-up.

Basic Path	1. The Student navigates to the sign-in page and enters their email address and password. 2. The system validates the credentials against the stored, hashed account record. 3. The system establishes an authenticated session for the Student. 4. The system directs the Student to their Dashboard.
Alternative Paths	If the Student does not yet have an account, they may create one via a self-service sign-up form (name, email address and password); this creates a new account with the Student role before proceeding to Login.
Postcondition	The Student has an authenticated session and is on their Dashboard.
Exception Paths	If the credentials are invalid, the system displays an error and the Student may retry or abandon the attempt.
Other	Every other Student use case in this document requires the Student to be authenticated via this use case first.

3.2.2 Register and Select Programme and Modules

Use Case Name	Register and Select Programme and Modules
XRef	Section 2.2.1, Register and Select Programme and Modules
Trigger	A newly registered Student chooses to select their academic programme and the modules they are registered for.
Precondition	The Student has created an account and is authenticated (Section 3.2.1, Login (Student)); programme and module data has been loaded into the system by a System Administrator.

Basic Path	1. The system presents a list of programmes sourced from the approved programme data. 2. The Student selects their programme. 3. The system presents the list of modules linked to the selected programme. 4. The Student selects the modules they are registered for. 5. The system records the selected programme and modules against the Student's profile.
Alternative Paths	If the Student's programme has not yet been loaded into the system, the Student is informed and directed to contact a System Administrator.
Postcondition	The Student's selected programme and modules are saved and available for assessment-structure configuration.
Exception Paths	The Student may abandon the selection at any time before submitting.
Other	Programme and module data displayed here is maintained by the System Administrator (Section 3.2.10, Manage Programme and Module Data).

3.2.3 Configure Assessment Structure

Use Case Name	Configure Assessment Structure
XRef	Section 2.2.1, Configure Assessment Structure
Trigger	The Student selects a registered module to configure its assessment structure.
Precondition	The Student has selected the module (Section 3.2.2).
Basic Path	1. The system presents the five assessment categories — Tests, Assignments, Quizzes, Practicals and Tutorials — for the selected module.

	2. The Student enters, for each category, its percentage contribution towards the module's DP progress, the total number of assessments expected in that category, and how many of those assessments will be recorded towards DP. 3. The system validates in real time that the entered category weightings sum to 100%. 4. The Student submits the configuration. 5. The system stores the configuration, excluding any category configured at 0% contribution from further DP calculations.
Alternative Paths	If the weightings do not sum to 100% when the Student submits, the system displays the current total and prevents submission until corrected.
Postcondition	The module's assessment structure is saved and used for all subsequent DP progress calculations for that module.
Exception Paths	The Student may abandon the configuration at any time before submitting.
Other	In the initial prototype, this configuration is entered by the Student. Enabling Academic Staff to perform this configuration on behalf of their students is a planned future enhancement, outside the scope of this document.

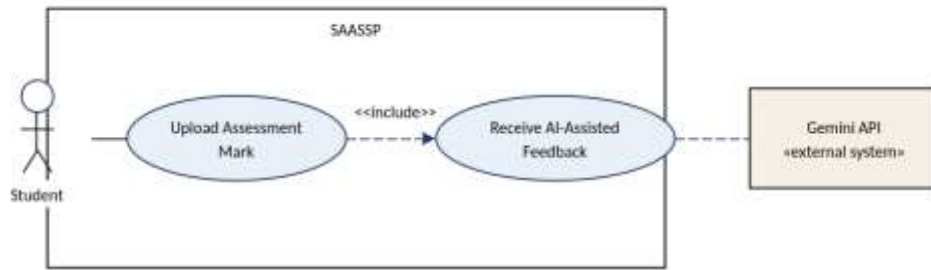
3.2.4 Upload Assessment Mark

Use Case Name	Upload Assessment Mark
XRef	Section 2.2.1, Upload Assessment Mark
Trigger	The Student receives a mark for a test, assignment, quiz, practical or tutorial and chooses to upload it.

Precondition	The Student has configured the assessment structure for the module (Section 3.2.3).
Basic Path	1. The Student selects the module and the assessment category. 2. The Student enters the percentage mark achieved. 3. The system validates that the entered mark is a valid percentage. 4. The system adds the mark to the category's full upload history and, if the category's recorded-mark cap has not yet been reached, adds it to the recorded pool; otherwise the system compares it with the lowest mark currently in the recorded pool and replaces that mark if the new mark is higher. 5. The system recalculates the module's overall DP progress percentage and threshold band. 6. The system invokes the Receive AI-Assisted Feedback use case (Section 3.2.5).
Alternative Paths	If the entered mark is outside the valid percentage range, the system rejects the entry and prompts the Student to correct it.
Postcondition	The mark is stored, the module's DP progress percentage and threshold band are updated, and feedback generation is triggered.
Exception Paths	The Student may abandon the upload at any time before submitting.
Other	A mark displaced from the recorded pool by a later, better mark remains in the category's full upload history and remains available for feedback-narrative purposes.

3.2.5 Receive AI-Assisted Feedback

Diagram:



Use Case Name	Receive AI-Assisted Feedback
XRef	Section 2.2.1; invoked by Section 3.2.4, Upload Assessment Mark
Trigger	A mark upload changes a module's DP progress percentage (invoked automatically by Section 3.2.4).
Precondition	The module's DP progress percentage and threshold band have just been recalculated.
Basic Path	1. The system determines the module's threshold band from the recalculated DP progress percentage, using the six fixed bands defined in Section 3.3.1. 2. The system determines the trend classification (increasing, dropping, fluctuating, or not yet available) for the assessment category just updated, from that category's full upload history. 3. The system assembles a structured feedback request containing the current DP progress percentage, threshold band, the category and mark just uploaded, the category's trend classification, and the category's full upload history. 4. The system sends the structured request to the Gemini API over HTTPS. 5. The Gemini API returns a generated feedback message. 6. The system stores and displays the feedback message to the Student.
Alternative Paths	If the Gemini API is unavailable or returns an error, the system displays the updated DP progress percentage and threshold band without a generated feedback message.

Postcondition	A feedback message is stored and displayed for the module.
Exception Paths	This use case cannot be initiated directly by a user; it is always invoked by Section 3.2.4.
Other	The threshold band and trend classification are determined entirely by SAASSP's own rule-based logic; the Gemini API is responsible only for the wording of the feedback message, never for the underlying figures or classifications.

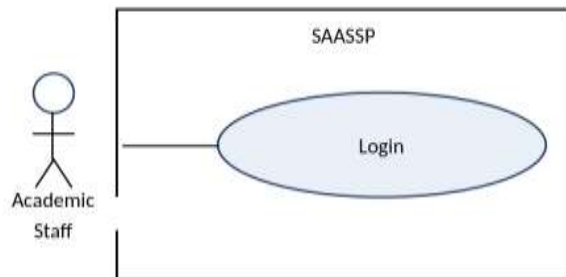
3.2.6 View DP Progress Dashboard

Use Case Name	View DP Progress Dashboard
XRef	Section 2.2.1, View DP Progress Dashboard
Trigger	The Student selects a module from their home screen to view its progress.
Precondition	The Student has configured the module's assessment structure (Section 3.2.3).
Basic Path	1. The system displays the module's current weighted DP progress percentage and its threshold band. 2. The system displays a per-category breakdown showing uploaded marks, which are currently recorded towards DP, and which have been displaced. 3. The system displays the most recent AI-assisted feedback message for the module.
Alternative Paths	If no marks have yet been uploaded for the module, the system displays the configured assessment structure with a DP progress percentage of zero.
Postcondition	None; this is a read-only use case.

Exception Paths	None.
Other	None.

3.2.7 Login (Academic Staff)

Diagram:



Use Case Name	Login (Academic Staff)
XRef	Section 2.2.2, Login (Academic Staff)
Trigger	An Academic Staff member with an existing account chooses to sign in.
Precondition	The Academic Staff member has a registered account, created by a System Administrator (Section 3.2.11, Manage User Accounts).
Basic Path	1. The Academic Staff member navigates to the sign-in page and enters their email address and password. 2. The system validates the credentials against the stored, hashed account record. 3. The system establishes an authenticated session for the Academic Staff member. 4. The system directs the Academic Staff member to their Students list.

Alternative Paths	None; Academic Staff accounts cannot be self-created.
Postcondition	The Academic Staff member has an authenticated session and is on their students list.
Exception Paths	If the credentials are invalid, the system displays an error and the Academic Staff member may retry or abandon the attempt.
Other	Section 3.2.8, View Student Progress, requires the Academic Staff member to be authenticated via this use case first.

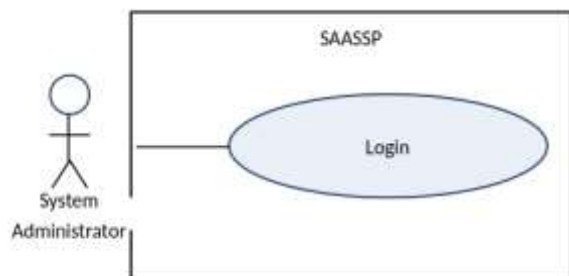
3.2.8 View Student Progress

Use Case Name	View Student Progress
XRef	Section 2.2.2, View Student Progress
Trigger	The Academic Staff member selects a student and module to review during an advising session.
Precondition	The Academic Staff member is authenticated, and the student has configured at least one module.
Basic Path	1. The system presents a list of students associated with the Academic Staff member. 2. The Academic Staff member selects a student and a module. 3. The system displays the module's DP progress percentage, threshold band, per-category breakdown and recent feedback, in read-only form.
Alternative Paths	None.
Postcondition	None; this is a read-only use case.

Exception Paths	None.
Other	Academic Staff cannot edit a student's assessment-structure configuration or marks in the initial prototype (see Section 1.2).

3.2.9 Login (Administrator)

Diagram:



Use Case Name	Login (Administrator)
XRef	Section 2.2.3, Login (Administrator)
Trigger	A System Administrator with an existing account chooses to sign in.
Precondition	The System Administrator has a registered account.
Basic Path	1. The System Administrator navigates to the sign-in page and enters their email address and password. 2. The system validates the credentials against the stored, hashed account record. 3. The system establishes an authenticated session for the System Administrator. 4. The system directs the System Administrator to the programme-data management screen.

Alternative Paths	None.
Postcondition	The System Administrator has an authenticated session and is on the programme-data management screen.
Exception Paths	If the credentials are invalid, the system displays an error and the System Administrator may retry or abandon the attempt.
Other	Sections 3.2.10 and 3.2.11 require the System Administrator to be authenticated via this use case first.

3.2.10 Manage Programme and Module Data

Use Case Name	Manage Programme and Module Data
XRef	Section 2.2.3, Manage Programme and Module Data
Trigger	The System Administrator selects to load or update programme and module data.
Precondition	The System Administrator is authenticated and has approved programme data available (programme codes and names, module codes and names, and the programme-module links).
Basic Path	1. The System Administrator uploads or enters the programme data. 2. The System Administrator uploads or enters the module data. 3. The System Administrator links modules to programmes. 4. The system validates that every linked module and programme code exists in the respective data, and stores the data.

Alternative Paths	If a referenced programme or module code does not exist, the system rejects that entry and reports it to the System Administrator.
Postcondition	The programme and module data is available for Student registration and module selection.
Exception Paths	The System Administrator may abandon the operation at any time.
Other	None.

3.2.11 Manage User Accounts

Use Case Name	Manage User Accounts
XRef	Section 2.2.3, Manage User Accounts
Trigger	The System Administrator selects to view, create, update, or deactivate a user account.
Precondition	The System Administrator is authenticated.
Basic Path	1. The system presents a list of user accounts with their assigned role (Student, Academic Staff or System Administrator). 2. The System Administrator selects an account to create, update or deactivate. 3. The system applies the change and updates the account record.
Alternative Paths	None.
Postcondition	The user account list reflects the change.
Exception Paths	The System Administrator may abandon the operation at any time.

Other	None.
--------------	-------

3.3 Detailed Non-Functional Requirements

3.3.1 Logical Structure of the Data

The logical structure of the data to be stored in the SAASSP MySQL database is given below. The six threshold bands referenced throughout this document are: 75–100% very safe/excellent, 70–74% safe, 60–69% generally safe, 50–59% pass but some risk, 40–49% danger/at risk of failing, and 0–39% critical/failing.

User Entity

Data Item	Type	Description	Comment
User ID	Integer	Unique identifier for the account	Primary key
Name	Text	Full name of the user	
Email Address	Text	Login and contact address	
Role	Category	Student, Academic Staff or System Administrator	
Password Hash	Text	Hashed account credential	Never stored in plain text

Programme Entity

Data Item	Type	Description	Comment
Programme Code	Text	Unique programme identifier	Primary key; sourced from approved programme data

Data Item	Type	Description	Comment
Programme Name	Text	Full name of the programme	

Module Entity

Data Item	Type	Description	Comment
Module Code	Text	Unique module identifier	Primary key; sourced from approved module data
Module Name	Text	Full name of the module	

Student Module Entity

Data Item	Type	Description	Comment
Student	Pointer	User entity of the Student	
Programme	Pointer	Programme entity selected by the Student	
Module	Pointer	Module entity selected by the Student	May be several per Student

Assessment Category Configuration Entity

Data Item	Type	Description	Comment
Student Module	Pointer	Student Module entity this configuration belongs to	
Category Name	Category	Tests, Assignments, Quizzes, Practicals or Tutorials	
DP Weighting	Integer	Percentage contribution towards DP progress	0 excludes the category from DP calculation
Total Assessments	Integer	Total assessments expected in the category	
Recorded Count	Integer	Number of assessments recorded towards DP	Caps the recorded-mark pool

Assessment Mark Entity

Data Item	Type	Description	Comment
Mark ID	Integer	Unique identifier for the uploaded mark	Primary key
Student Module	Pointer	Student Module entity this mark belongs to	
Category Name	Category	Assessment category the mark belongs to	
Percentage Achieved	Decimal	Mark achieved, as a percentage	

Data Item	Type	Description	Comment
Date Uploaded	Date	Date and time the mark was uploaded	
Recorded Towards DP	Boolean	Whether the mark currently counts towards DP	False once displaced by a later, better mark

Feedback Message Entity

Data Item	Type	Description	Comment
Feedback ID	Integer	Unique identifier for the feedback message	Primary key
Mark	Pointer	Assessment Mark entity that triggered this feedback	
DP Progress Percentage	Decimal	DP progress percentage at the time of generation	
Threshold Band	Category	One of the six fixed threshold bands	Determined by system logic
Trend Classification	Category	Increasing, dropping, fluctuating, or not yet available	Determined by system logic
Generated Text	Text	The feedback message returned by the Gemini API	

3.3.2 Security

Access to SAASSP is role-based: a Student may only view and edit their own assessment configuration, marks and feedback; an Academic Staff member has read-only access to the students associated with them; a System

Administrator may maintain programme, module and user-account data but does not directly edit a Student's marks or configuration.

All communication between the browser, the SAASSP web application and the Gemini API uses HTTPS. Passwords are stored as salted hashes and are never transmitted or stored in plain text. The Gemini API key is held server-side only and is never exposed to the browser.

Student-entered marks are labelled as provisional and self-reported throughout the system, and are never presented as an official academic record.



Department of Computer Science

Software Design Description

Project Title:

SMART ACADEMIC ADVISORY & STUDENT SUCCESS PLATFORM (SAASSP)

Supervisor:

Mrs IN Ezeji

Submitted by:

Student Name	Student Number
Nhlanhlehle Ngcobo	240035192
Celimpilo Dlamini	202020444
Zinhle Mhlongo	202285904
Mphilisi Hlandze	230013633
Nokwanda Sibiya	230015229
Thandiwe Mvelase	240111090

Date: 25/09/2026

Table of Contents

1.0. Introduction

- 1.1. Purpose
- 1.2. Scope
- 1.3. Glossary
- 1.4. References
- 1.5. Overview of Document

2.0. Deployment Diagram

3.0. Architecture Design

- 3.1 Web System Architecture

4.0. Data Structure Design

- 4.1 Data Field Types and Sizes

5.0. Use Case Realizations

6.0. Interface Design

7.0. Help System Design

Table of Figures

Figure 1 Deployment Diagram

Figure 2 Architecture Design

Figure 3 System Sequence Diagram

Figure 4 Login Sequence Diagram

Figure 5 Configure Assessment Structure Sequence Diagram

Figure 6 Upload Assessment Mark & Receive AI-Assisted Feedback Sequence Diagram

Figure 7 View Student Progress Sequence Diagram

Figure 8 Manage Programme and Module Data Sequence Diagram

Figure 9 Interface: Home Page

Figure 10 Create account

Figure 11 & 12 All Programmes and Modules

Figure 13 Each student modules

Figure 14 & 15 Each module dashboard

1.0. Introduction

1.1. Purpose

This Software Design Description provides the design details of the Smart Academic Advisory & Student Success Platform (SAASSP), specifically its Duly-Performed (DP) Progress & AI-Assisted Feedback module.

The expected audience is the project supervisor, Mrs IN Ezeji, the SAASSP development team, and any future maintainer of the platform. It also serves as a design reference alongside the SAASSP Software Requirements Specification (SRS).

1.2. Scope

This document contains a description of the design of SAASSP's core student-facing functionality: authentication, programme and module selection, assessment-structure configuration, mark upload, continuous weighted DP progress calculation, trend classification, and AI-assisted feedback generation, together with the read-only Academic Staff view and the System Administrator's data-management functions.

The basic architecture is a layered web application following a client-server paradigm: a Flask (Python) application server renders HTML5/CSS3/Bootstrap 5 pages to the browser and communicates with a MySQL database and, for feedback generation only, the external Gemini API.

This document does not cover module recommendations, prerequisite or co-requisite checking, credit accumulation or graduation-progress tracking, or machine-learning-based risk classification, none of which are part of the current system scope. Lecturer-side assessment-structure configuration and a conversational AI Academic Advisor chatbot are noted as planned future enhancements only, and are not designed in this document.

1.3. Glossary

Term	Definition
API	Application Programming Interface.
DP	Duly Performed; the continuous academic standing a student must maintain in a module's assessed coursework.

Term	Definition
Gemini API	Google's large language model API, used to generate the wording of AI-assisted feedback messages.
IEEE	Institute of Electrical and Electronic Engineers.
ORM	Object-Relational Mapping; used to map Python objects to MySQL database records.
SAASSP	Smart Academic Advisory & Student Success Platform; the system described by this document.
SDD	Software Design Description; this document.
SRS	Software Requirements Specification.
Threshold Band	One of six fixed bands that a DP progress percentage is mapped to.
Trend Classification	A per-category classification (increasing, dropping or fluctuating) derived from a student's mark-upload history.
WSGI	Web Server Gateway Interface; the standard Python interface between Flask and the application server.

1.4. References

- IEEE. IEEE Std 830-1998 IEEE Recommended Practice for Software Requirements Specifications. IEEE Computer Society, 1998.
- SAASSP Group 20. Software Requirements Specification: Smart Academic Advisory & Student Success Platform. University of Zululand, 2026.
- Laudon, K. C., & Laudon, J. P. (2022). Management information systems: Managing the digital firm (17th ed.). Pearson.

- Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill.
- Sommerville, I. (2021). Software engineering (10th ed.). Pearson.

1.5. Overview of Document

Section 2 is a Deployment Diagram that shows the physical nodes on which SAASSP resides and how they communicate.

Section 3 is the Architecture Design, which specifies the design components that collaborate to perform SAASSP's functions. Each component has a description of the services it provides, expanded into a set of operations.

Section 4 concerns the Data Structure Design.

Section 5 contains the Use Case Realizations. Each use case stated in the SRS is traced here through sequence diagrams showing the collaborating design components.

Section 6 discusses the User Interface Design.

Section 7 covers the help system.

2.0. Deployment Diagram

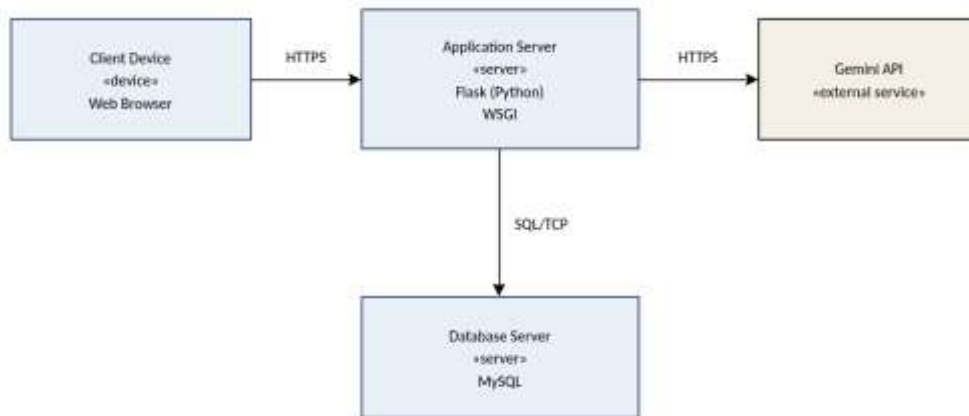


Figure 1 Deployment Diagram

SAASSP is deployed across three physical/logical nodes. The client device runs a standard web browser and communicates with the application server over HTTPS. The application server runs the Flask (Python) application under WSGI and is the only component that communicates directly with the database server (MySQL, over SQL/TCP) and with the Gemini API (over HTTPS). The Gemini API is an external service and is never accessed directly by the client device.

3.0. Architecture Design

3.1. Web System Architecture

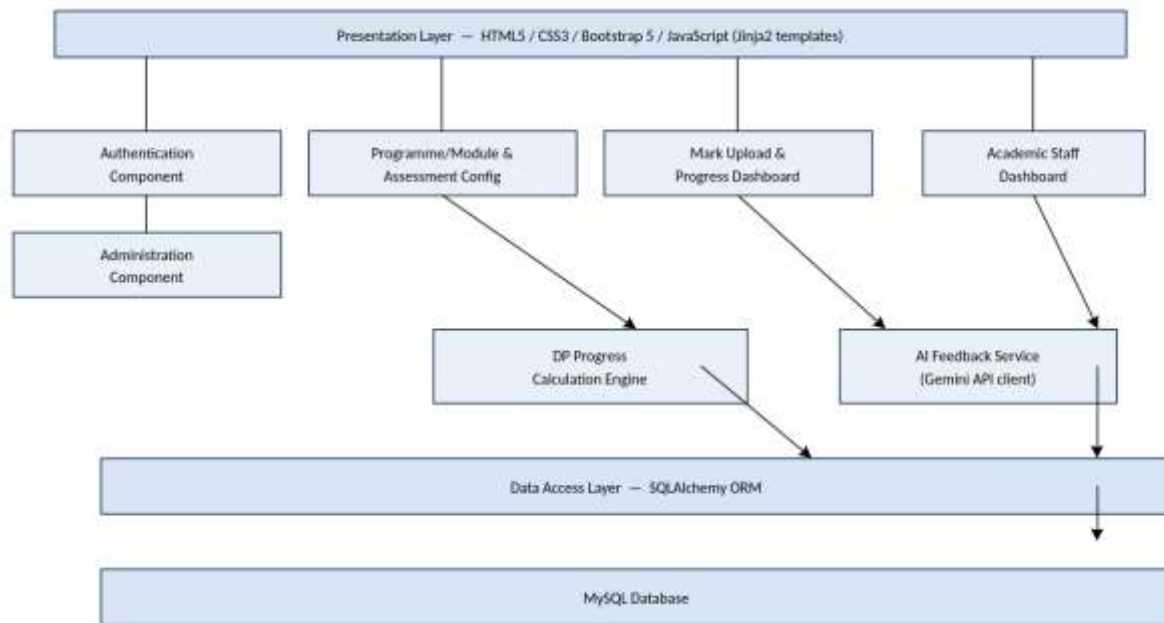


Figure 2 Architecture Design

SAASSP follows a layered architecture. The Presentation Layer renders server-side templates to the browser. Eight design components, described below, implement the application's functionality and sit above a Data Access Layer (SQLAlchemy ORM) that mediates all access to the MySQL database. The DP Progress Calculation Engine and AI Feedback Service are backend services rather than web pages, and are deliberately kept independent of one another: the engine never calls the feedback service, and the feedback service never recalculates progress.

Authentication Component

Name: Authentication Component

Type: Web page / Service

Description: Presents the sign-in form and validates credentials for all three actor types (Student, Academic Staff, System Administrator), establishing an authenticated session and directing the user to their role's home screen. Also presents the self-service sign-up form for new Student accounts.

Attributes: None

Resources: User entity (Database)

Operations:

Name: Login()

Arguments: email: String, password: String

Returns: Authenticated session, or an error message

Pre-condition: The user has a registered account.

Post-condition: An authenticated session is established and the user is directed to their role's home screen.

Exceptions: Invalid credentials.

Flow of Events:

1. The user navigates to the sign-in page and enters their email address and password.
2. The system validates the credentials against the stored, hashed account record.
3. The system establishes an authenticated session for the user.
4. The system directs the user to the home screen appropriate to their role.

Name: Register()

Arguments: name: String, email: String, password: String

Returns: New account created, or an error message

Pre-condition: No account exists for the given email address.

Post-condition: A new User record is created with the Student role.

Exceptions: Email address already registered.

Flow of Events:

1. The prospective Student completes the self-service sign-up form.
2. The system checks that the email address is not already registered.
3. The system creates a new User record with the Student role and a hashed password.
4. The Student proceeds to Login().

Programme, Module & Assessment Configuration Component

Name: Programme, Module & Assessment Configuration Component

Type: Web page

Description: Lets a student select their programme and modules from approved programme data, and configure each selected module's assessment categories (weightings, total assessments and recorded-mark counts).

Attributes: None

Resources: Programme, Module, Student Module and Assessment Category Configuration entities (Database)

Operations:

Name: `SelectProgrammeAndModules()`

Arguments: programmeCode: String, moduleCodes: List<String>

Returns: None

Pre-condition: The Student is authenticated.

Post-condition: Student Module records are created for each selected module.

Exceptions: None.

Flow of Events:

1. The system presents a list of programmes sourced from the approved programme data.
2. The Student selects their programme.
3. The system presents the list of modules linked to the selected programme.
4. The Student selects the modules they are registered for.
5. The system records the selected programme and modules against the Student's profile.

Name: `ConfigureAssessmentStructure()`

Arguments: moduleCode: String, categories: List<{name, weight, total, recorded}>

Returns: Saved configuration, or a validation error

Pre-condition: The Student has selected the module.

Post-condition: The module's Assessment Category Configuration records are saved.

Exceptions: Category weightings do not sum to 100%.

Flow of Events:

1. The system presents the five assessment categories for the selected module.
2. The Student enters, for each category, its DP weighting, total assessments and recorded count.
3. The system validates in real time that the weightings sum to 100%.
4. The Student submits the configuration.
5. The system stores the configuration, excluding any category configured at 0% contribution.

Mark Upload & DP Progress Dashboard Component

Name: Mark Upload & DP Progress Dashboard Component

Type: Web page

Description: Lets a Student upload a mark for a configured assessment category, and displays a module's current DP progress percentage, threshold band, per-category breakdown, and most recent AI-assisted feedback.

Attributes: None

Resources: Assessment Mark and Feedback Message entities (Database); DP Progress Calculation Engine; AI Feedback Service

Operations:

Name: UploadMark()

Arguments: moduleCode: String, category: String, percentage: Decimal

Returns: Updated DP progress percentage and feedback message

Pre-condition: The module's assessment structure is configured.

Post-condition: The mark is stored; the module's DP progress and feedback are updated.

Exceptions: Mark outside valid percentage range.

Flow of Events:

1. The Student selects the module and assessment category and enters the percentage achieved.
2. The system validates the mark and adds it to the category's upload history.
3. The system requests DP Progress Calculation Engine.CalculateProgress() and .ClassifyTrend().
4. The system requests AI Feedback Service.GenerateFeedback() with the recalculated data.
5. The system stores the mark and the returned feedback message.
6. The system displays the updated DP progress and feedback to the Student.

Name: ViewDashboard()

Arguments: moduleCode: String

Returns: DP progress percentage, threshold band, category breakdown, feedback

Pre-condition: None.

Post-condition: None; this operation is read-only.

Exceptions: None.

Flow of Events:

1. The Student selects a module from their home screen.
2. The system displays the module's current DP progress percentage, threshold band and per-category breakdown.
3. The system displays the most recent AI-assisted feedback message.

DP Progress Calculation Engine

Name: DP Progress Calculation Engine

Type: Service (backend business logic)

Description: Deterministic, rule-based service responsible for maintaining each assessment category's recorded-mark pool, calculating a module's weighted DP progress percentage, mapping it to one of six threshold bands, and classifying the trend of marks within a category. This engine never depends on the AI Feedback Service.

Attributes: None

Resources: Assessment Mark and Assessment Category Configuration entities (Database)

Operations:

Name: CalculateProgress()

Arguments: moduleCode: String

Returns: DP progress percentage, threshold band

Pre-condition: The module has at least one configured, non-zero-weighted category.

Post-condition: None.

Exceptions: None.

Flow of Events:

1. For each non-zero-weighted category, retrieve the recorded-mark pool (the best min(cap, marksWritten) marks).
2. Sum the DP points earned across every recorded mark.
3. Sum the DP points attempted (the weighting share represented by each recorded mark).
4. Divide points earned by points attempted and express the result as a percentage.
5. Map the resulting percentage to one of the six fixed threshold bands.

Name: ClassifyTrend()

Arguments: moduleCode: String, category: String

Returns: Trend classification (increasing, dropping, fluctuating, or not yet available)

Pre-condition: None.

Post-condition: None.

Exceptions: None.

Flow of Events:

1. Retrieve the category's full, ordered mark-upload history.
2. If fewer than three marks exist, return "not yet available".
3. Compare each consecutive pair of marks for direction.
4. If all comparisons agree, or exactly one disagrees, classify as increasing or dropping in the dominant direction.
5. If two or more comparisons disagree, classify as fluctuating.

AI Feedback Service

Name: AI Feedback Service

Type: Service (Gemini API client)

Description: Assembles a structured feedback request from the DP Progress Calculation Engine's output and sends it to the Gemini API, returning the generated feedback text for storage and display. This service never determines the DP progress percentage, threshold band or trend classification itself.

Attributes: None

Resources: Feedback Message entity (Database); Gemini API (external service)

Operations:

Name: GenerateFeedback()

Arguments: moduleCode, category, mark, progressPercentage, thresholdBand, trendClassification, categoryHistory

Returns: Generated feedback text, or no feedback if the API is unavailable

Pre-condition: The module's DP progress percentage and threshold band have just been recalculated.

Post-condition: A Feedback Message record is stored.

Exceptions: Gemini API unavailable or returns an error.

Flow of Events:

1. The service assembles a structured HTTPS request containing the current DP progress percentage, threshold band, the category and mark just uploaded, the trend classification, and the category's full upload history.
2. The service sends the request to the Gemini API.
3. The Gemini API returns generated feedback text.
4. The service stores the feedback message and returns it to the calling component.

Academic Staff Dashboard Component

Name: Academic Staff Dashboard Component

Type: Web page

Description: Presents a read-only view of a selected student's DP progress, per-category breakdown and recent feedback for a selected module.

Attributes: None

Resources: Assessment Mark and Feedback Message entities (Database, read-only)

Operations:

Name: ViewStudentProgress()

Arguments: studentId: Integer, moduleCode: String

Returns: DP progress percentage, threshold band, category breakdown, feedback (read-only)

Pre-condition: The Academic Staff member is authenticated and associated with the student.

Post-condition: None.

Exceptions: None.

Flow of Events:

1. The Academic Staff member selects a student from their associated list.

2. The Academic Staff member selects a module.
3. The system displays the module's DP progress percentage, threshold band, per-category breakdown and recent feedback, in read-only form.

Administration Component

Name: Administration Component

Type: Web page

Description: Lets a System Administrator load and maintain approved programme and module data, and manage user accounts and roles.

Attributes: None

Resources: Programme, Module and User entities (Database)

Operations:

Name: ManageProgrammeAndModuleData()

Arguments: programme and module records

Returns: Saved data, or validation errors

Pre-condition: The System Administrator is authenticated.

Post-condition: The programme and module data is available for Student registration and module selection.

Exceptions: A referenced programme or module code does not exist.

Flow of Events:

1. The System Administrator uploads or enters the programme data.
2. The System Administrator uploads or enters the module data.
3. The System Administrator links modules to programmes.
4. The system validates and stores the data.

Name: ManageUserAccounts()

Arguments: user account details, role

Returns: Updated account list

Pre-condition: The System Administrator is authenticated.

Post-condition: The account record is created, updated or deactivated.

Exceptions: None.

Flow of Events:

1. The system presents a list of user accounts with their assigned role.
2. The System Administrator selects an account to create, update or deactivate.
3. The system applies the change and updates the account record.

SAASSP Database

Name: SAASSP Database

Type: Database

Description: The SAASSP database resides on the application server and is created and maintained in MySQL. It stores all programme, module, assessment-configuration, mark and feedback data described in Section 4.

Attributes: See Section 4.1, Data Field Types and Sizes

Resources: None

Operations:

Name: New()

Arguments: None

Returns: None

Pre-condition: None.

Post-condition: A new record is added to the relevant table.

Exceptions: None.

Flow of Events:

1. The application submits a validated record.
2. The database adds the record to the relevant table.

Name: Replace()

Arguments: None

Returns: None

Pre-condition: The requested record exists.

Post-condition: The requested record is replaced with the updated record.

Exceptions: None.

Flow of Events:

1. The application submits an updated record.
2. The database overwrites the existing record with the updated record.

Name: Search()

Arguments: None

Returns: Record(s)

Pre-condition: None.

Post-condition: The database has been searched.

Exceptions: None.

Flow of Events:

1. The application queries the database for matching records.
2. If records are found, the database returns them to the application.

4.0. Data Structure Design

The data is stored in a relational database using MySQL. The fields for transmitting to and from the database are given in the following table.

4.1. Data Field Types and Sizes

Attribute Name	Attribute Type	Attribute Size
UserID*#^	Integer	11
Name*	String	100
Email*#	String	100
Role*#	String	20
PasswordHash*^	String	255
ProgrammeCode*#	String	10
ProgrammeName*	String	150
ModuleCode*#	String	10
ModuleName*	String	150
CategoryName*#	String	20
DPWeighting*	Integer	3
TotalAssessments*	Integer	3
RecordedCount*	Integer	3
MarkID#^	Integer	11

Attribute Name	Attribute Type	Attribute Size
PercentageAchieved*	Decimal	5,2
DateUploaded^	DateTime	-
RecordedTowardsDP^	Boolean	1
FeedbackID#^	Integer	11
ThresholdBand^	String	30
TrendClassification^	String	20
GeneratedText^	Text	-

Fields marked with an '*' are required fields. Fields marked with a '#' are used as a primary or foreign key for lookups. Fields marked with a '^' are system-generated or system-calculated and are never directly entered by a user.

5.0. Use Case Realizations

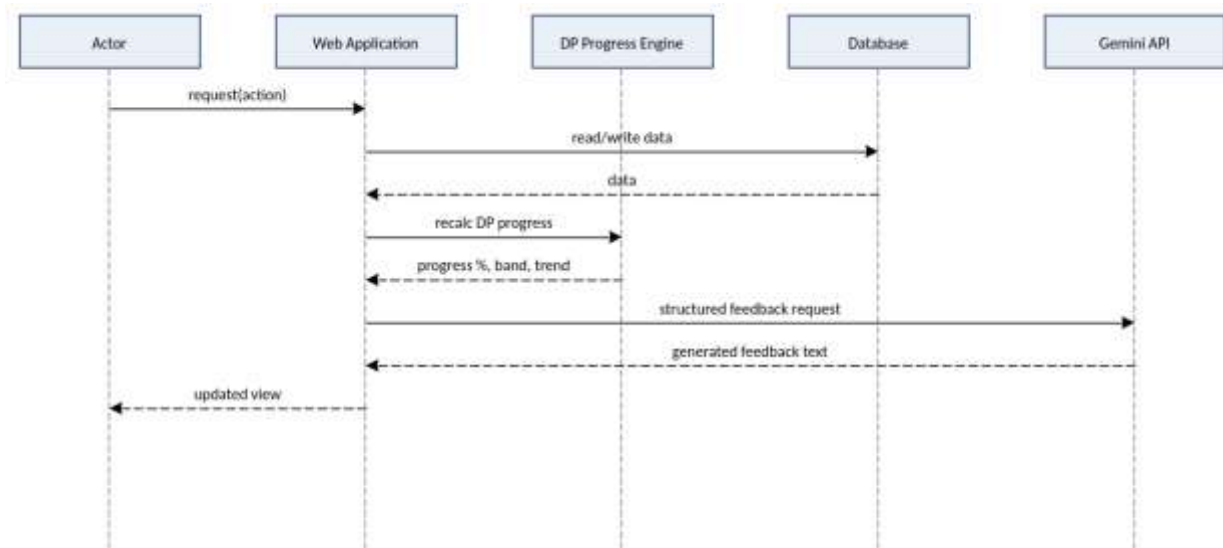


Figure 3 System Sequence Diagram

The diagram above shows the general interaction pattern common to SAASSP's actor-triggered use cases: the Web Application coordinates with the DP Progress Calculation Engine, the Database, and, where feedback generation applies, the Gemini API. The use case realizations below show this pattern applied to five representative use cases, one for each actor and the core AI-feedback flow.

Use Case: Login

See also Login in the Software Requirements Specification, Section 3.2.1.

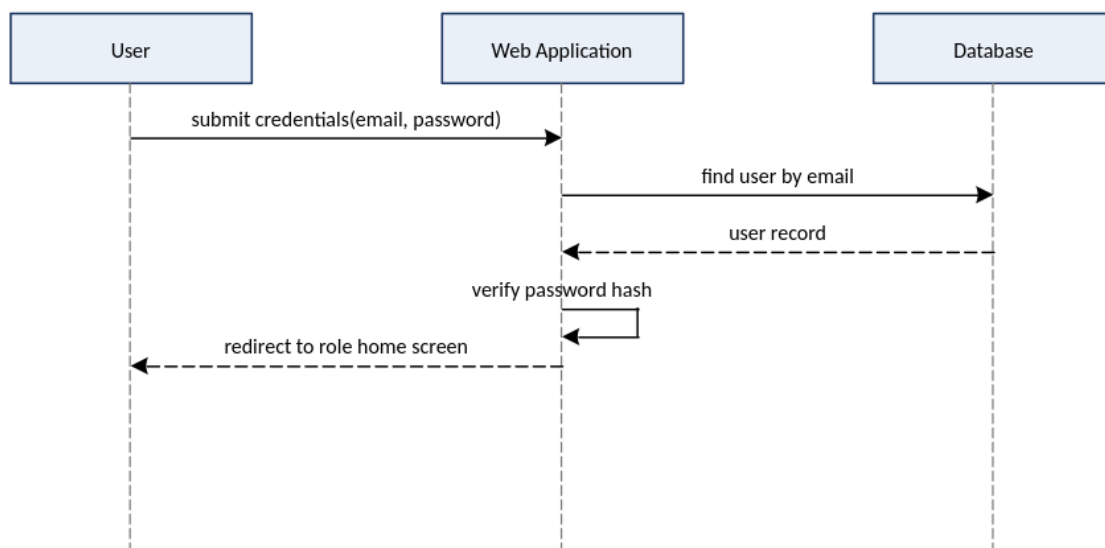


Figure 4 Login Sequence Diagram

Use Case: Configure Assessment Structure

See also Configure Assessment Structure in the Software Requirements Specification, Section 3.2.3.

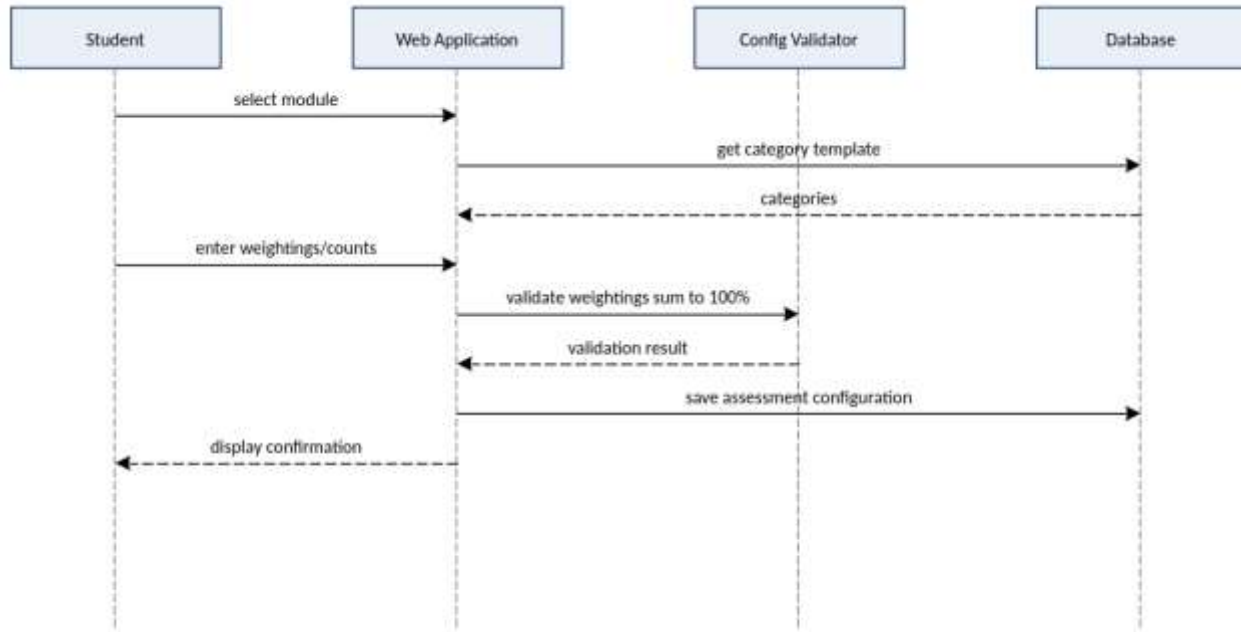


Figure 5 Configure Assessment Structure Sequence Diagram

Use Case: Upload Assessment Mark & Receive AI-Assisted Feedback

See also Upload Assessment Mark & Receive AI-Assisted Feedback in the Software Requirements Specification, Sections 3.2.4 and 3.2.5.

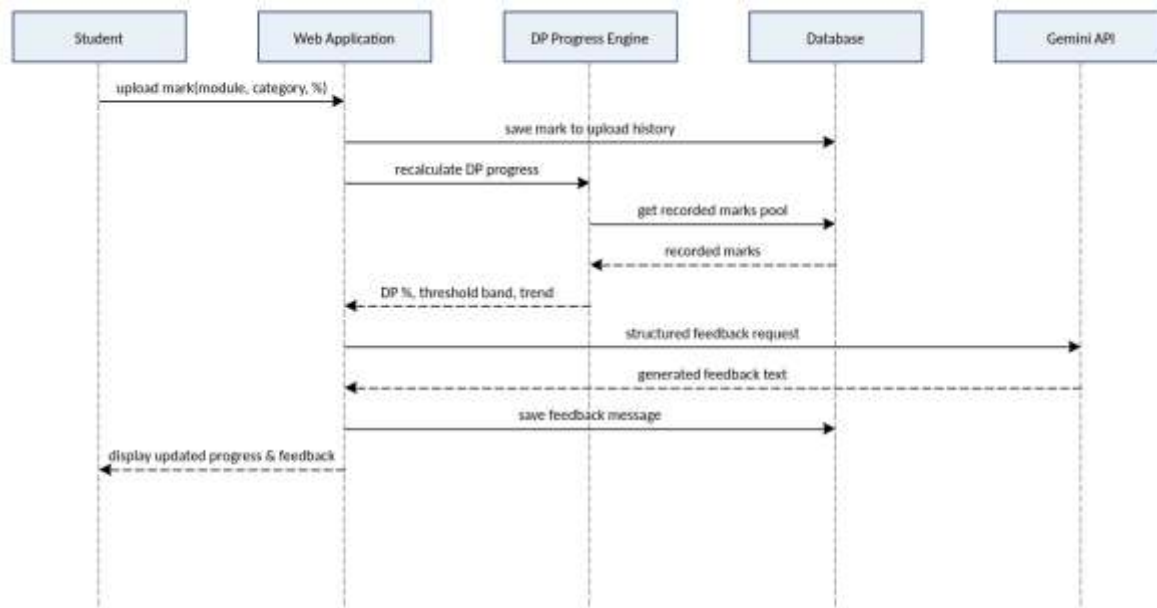


Figure 6 Upload Assessment Mark & Receive AI-Assisted Feedback Sequence Diagram

Use Case: View Student Progress

See also View Student Progress in the Software Requirements Specification, Section 3.2.7.

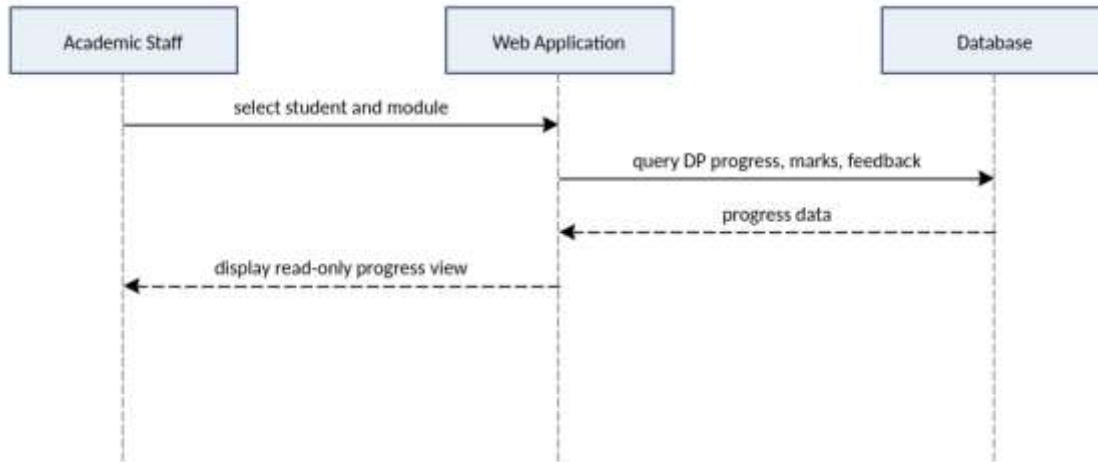


Figure 7 View Student Progress Sequence Diagram

Use Case: Manage Programme and Module Data

See also Manage Programme and Module Data in the Software Requirements Specification, Section 3.2.8.

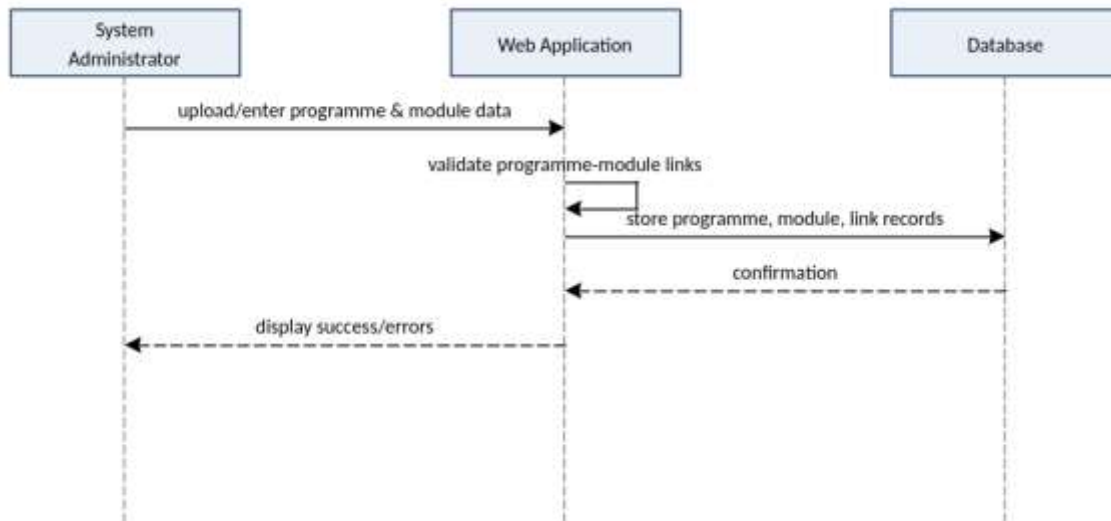


Figure 8 Manage Programme and Module Data Sequence Diagram

6.0. Interface Design

The interface will use the University of Zululand's official colour scheme (navy blue, gold, white and black) and Bootstrap 5 components to create a consistent, accessible experience across the Student, Academic Staff and System Administrator views. Every page follows a shared base layout with a UNIZULU-branded header and consistent navigation appropriate to the signed-in user's role.

Category-weighting entry on the assessment-configuration page shows a running total as the Student types, so an invalid configuration (not summing to 100%) is visibly flagged before submission is attempted. The mark-upload page shows the module's current DP progress percentage and threshold band immediately after a mark is submitted, followed by the AI-assisted feedback message once it has been generated.

Wireframes and final page designs will be added to a future revision of this document once the frontend implementation is finalised.

Figure 9: Home page

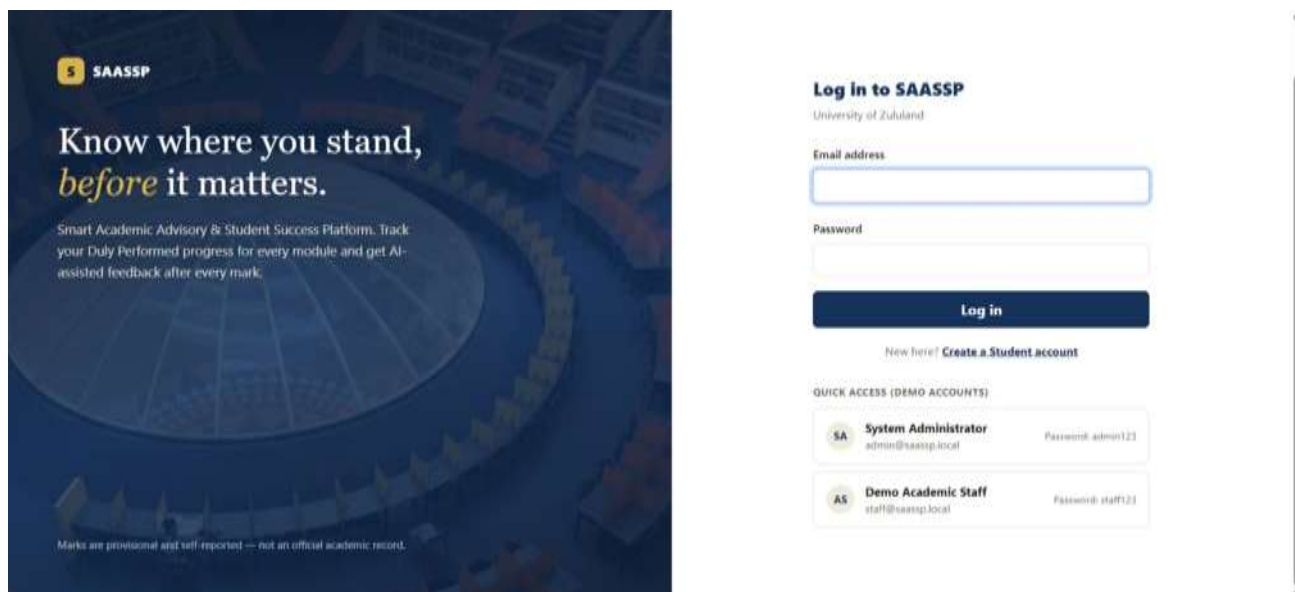


Figure 10: Create account

SAASSP

**Know where you stand,
before it matters.**

Create your Student account to select your programme and modules, and start tracking your Duty Performed progress.

Create your Student account
University of Zululand

Full name

Email address

Password Please fill out this field.

Create account

Already have an account? [log in](#)

Figure 11 & 12: All Programmes and Modules

SAASSP
Student Success Platform

Dashboard
My Modules

Select your programme

Choose the programme you are registered for. This list is sourced from the university's approved programme data.

- ☐ 4BSC01
APPLIED MATHEMATICS AND COMPUTER SCIENCE
- ☐ 4BSC02
APPLIED MATHEMATICS AND HYDROLOGY
- ☐ 4BSC03
APPLIED MATHEMATICS AND MATHEMATICS
- ☐ 4BSC04
APPLIED MATHEMATICS AND PHYSICS
- ☐ 4BSC05
APPLIED MATHEMATICS AND STATISTICS
- ☐ 4BSC06
BIOCHEMISTRY AND BOTANY

Figure 12:

Online Manager
Student
Log out

HYDROLOGY AND PHYSICS

48SC32
HYDROLOGY AND STATISTICS

48SC33
HYDROLOGY AND ZOOLOGY

48SC34
MATHEMATICS AND PHYSICS

48SC35
MATHEMATICS AND STATISTICS

48SC27
MICROBIOLOGY AND HUMAN MOVEMENT SCIENCE

48SC36
MICROBIOLOGY AND ZOOLOGY

Continue to modules

Figure 13: Each student modules

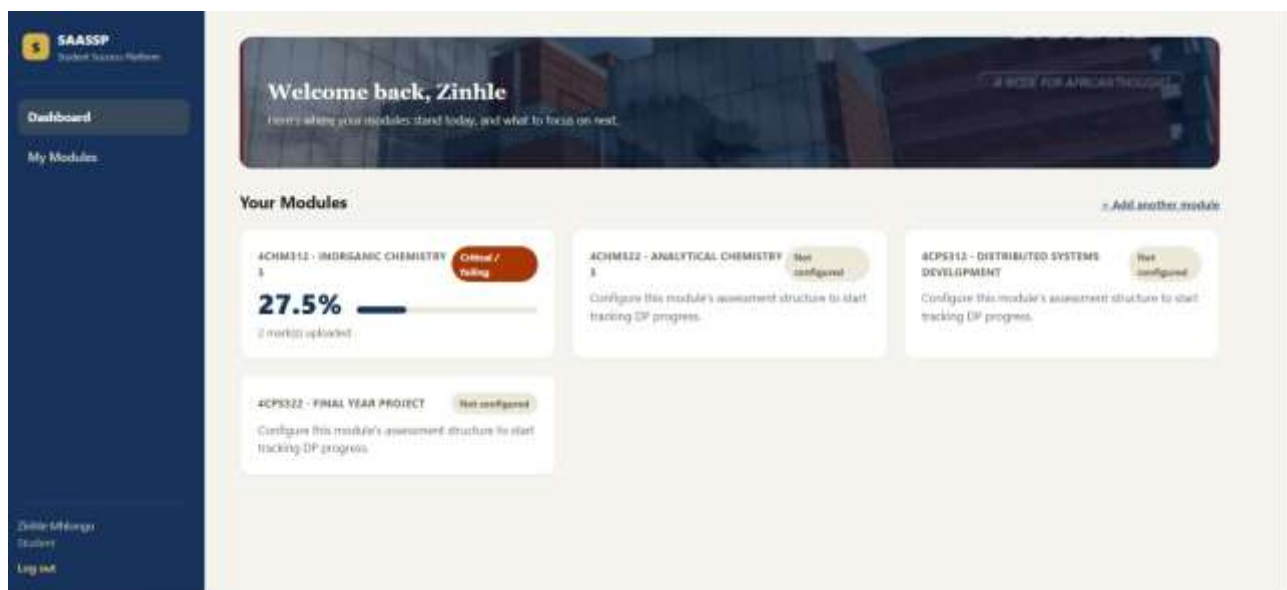


Figure 14 & 15: Each module dashboard

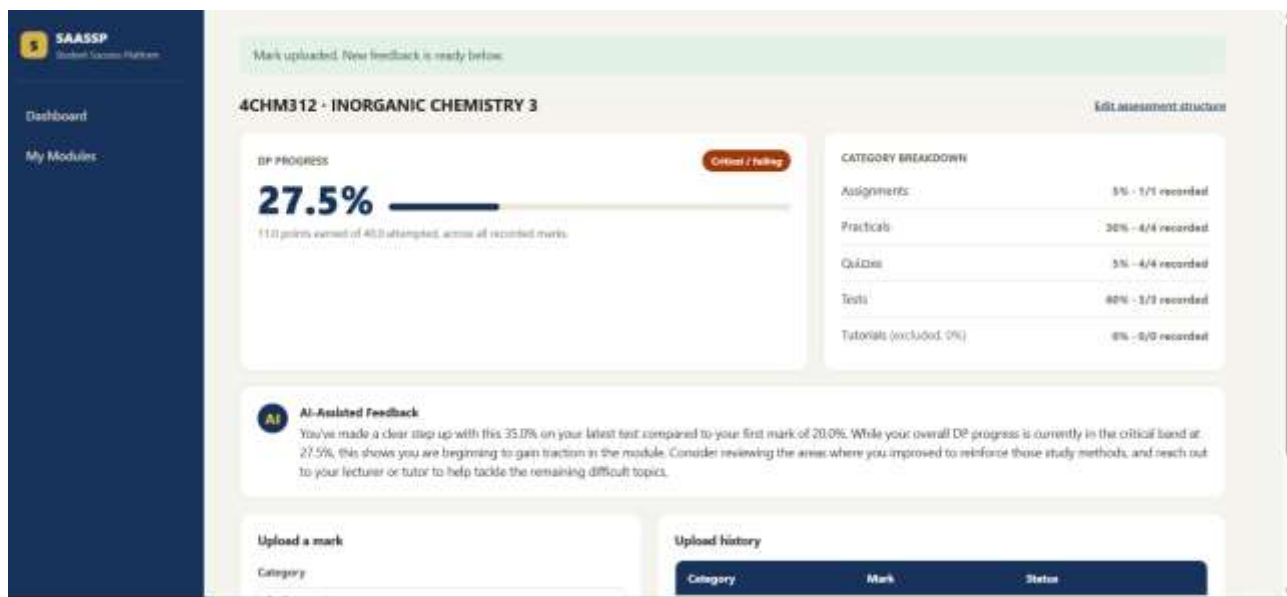
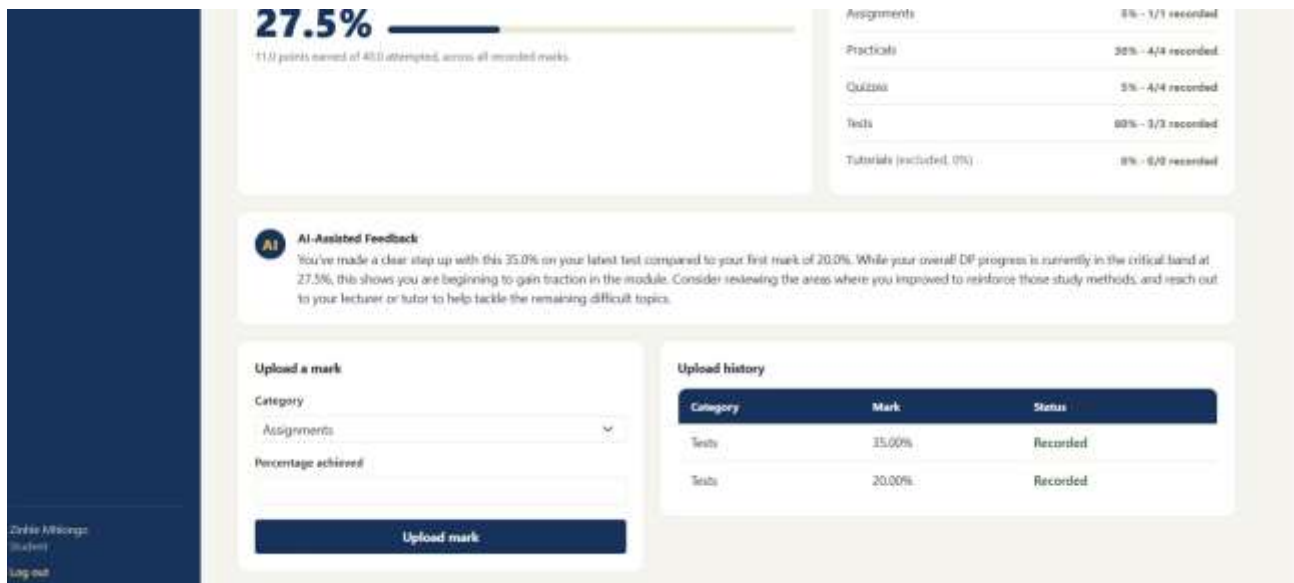


Figure 15



7.0. Help System Design

At this time there is no dedicated in-application help system planned. Guidance on configuring a module's assessment structure is provided online on the configuration page itself (through the real-time weighting-total indicator), and AI-assisted feedback messages are intended to provide the student with ongoing, contextual guidance after each mark upload in place of a separate static help system.



Department of Computer Science

Test Plan

Project Title:

SMART ACADEMIC ADVISORY & STUDENT SUCCESS PLATFORM (SAASSP)

Supervisor:

Mrs IN Ezeji

Submitted by:

Student Name	Student Number
Nhlanhlehle Ngcobo	240035192
Celimpilo Dlamini	202020444
Zinhle Mhlongo	202285904
Mphilisi Hlandze	230013633
Nokwanda Sibiya	230015229
Thandiwe Mvelase	240111090

Date: 25/09/2026

References

Documents that support this test plan include the SAASSP Project Proposal, the SAASSP Software Requirements Specification (SRS), and the SAASSP Software Design Description (SDD).

Introduction

This test plan for the SAASSP DP Progress & AI-Assisted Feedback module supports the following objectives:

- To define the tools and test levels to be used throughout the testing process.
- To communicate to the project team the items to be tested, set expectations around schedule, and define environmental needs.
- To define how the tests will be conducted, including the deterministic DP-calculation logic that must be verified independently of the AI-generated feedback wording.

Test Items

The systems to be tested are the components of the SAASSP web application described in the SDD: the Authentication Component; the Programme, Module & Assessment Configuration Component; the Mark Upload & DP Progress Dashboard Component; the DP Progress Calculation Engine; the AI Feedback Service (Gemini API client); the Academic Staff Dashboard Component; and the Administration Component, together with the underlying MySQL database.

The web-facing components should be tested in the latest stable versions of Chrome, Firefox and Microsoft Edge, on a Windows machine. The DP Progress Calculation Engine and AI Feedback Service are backend services and are tested independently of the browser, using automated unit and integration tests.

Features To Be Tested

Features to be tested include the following:

- As a Student, registering a new account and logging in.
- As a Student, selecting a programme and modules from the approved programme data.
- As a Student, configuring a module's assessment categories (DP weighting, total assessments, recorded count per category).
- As a Student, being prevented from submitting an assessment configuration whose category weightings do not sum to 100%.
- As a Student, uploading an assessment mark and having it validated as a percentage.

- As a Student, having a later, better mark correctly displace the lowest currently recorded mark once a category's recorded cap is reached.
- As a Student, viewing a module's current DP progress percentage and threshold band.
- As a Student, viewing a per-category breakdown of uploaded and recorded marks.
- As a Student, receiving AI-assisted feedback after every mark upload, including the first ever upload for a module.
- As a Student, having a category's trend correctly classified as increasing, dropping, fluctuating, or not yet available.
- As Academic Staff, viewing a student's DP progress, category breakdown and recent feedback in read-only form.
- As a System Administrator, loading and maintaining programme and module data.
- As a System Administrator, creating, updating and deactivating user accounts.

Features Not To Be Tested

The following are outside the current system scope and will not be tested: module recommendations and prerequisite/co-requisite checking; credit accumulation and graduation-progress tracking; machine-learning-based academic-risk classification; lecturer-side assessment-structure configuration; and the conversational AI Academic Advisor chatbot. All five are either out of scope or noted in the SRS and SDD as planned future enhancements only.

Native mobile application testing will not be performed; only responsive web-browser behaviour is tested.

Approach

Testing will be conducted at three levels, consistent with the SRS and SDD:

- Unit testing (Pytest) of the DP Progress Calculation Engine's CalculateProgress() and ClassifyTrend() operations, and of threshold-band mapping, run against predefined student profiles with known expected outcomes.
- Integration testing of the interactions between the web application, the database, and the Gemini API, including the case where the Gemini API is unavailable.
- Functional and scenario-based testing of each feature listed above, performed manually against documented test cases and tracked to Pass / Fail / Skip, with notes on actual results.

When a test case is marked as Fail, a bug report will be created in the project's GitHub Issues, consistent with the version-control and collaboration approach described in the Project Proposal.

Because AI-generated feedback wording is not deterministic, test cases for the AI Feedback Service verify that the correct structured data (DP progress percentage, threshold band, trend classification) is sent to the Gemini API and that a non-empty feedback message is stored and displayed, rather than asserting exact wording.

Pass/Fail Criteria

All core functionality must function as specified in the SRS and SDD. DP progress percentages, threshold bands and trend classifications calculated for every predefined test profile must exactly match their documented expected values, since this logic is deterministic. There must be no critical defects, and a Student must be able to complete the full cycle of configuring a module, uploading a mark, viewing updated progress, and receiving feedback without errors. At least 95% of all test cases should pass, and no failed case may be one that blocks this core cycle.

Suspension Criteria

Testing should be paused immediately if login fails for any role, if the DP Progress Calculation Engine produces an incorrect result for any predefined test profile, or if a basic Create, Read or Update action (assessment configuration, mark upload, or programme/module data entry) fails.

Test Deliverables

Upon completion, the executed test cases, their Pass/Fail/Skip results, and any linked GitHub Issues will be compiled into a test summary report by the team member responsible for testing that cycle.

Testing Tasks

The following activities must be completed:

- Test plan prepared.
- Test cases and predefined student test profiles (with known expected DP outcomes) prepared.
- Test environment ready, including test programme/module data and test accounts for each role.
- Unit and integration tests executed.
- Functional and scenario-based tests executed.
- Test summary report prepared.

Environmental Needs

The test database must be populated with the approved programme and module data (programme codes and names, module codes and names, and their links) and with a set of predefined synthetic student profiles covering each DP threshold band, including profiles designed to exercise the best-N-of-M mark-displacement rule and each trend classification. A test account for each role (Student, Academic Staff, System Administrator) is required. Where live Gemini API calls would be slow, costly or rate-limited, feedback-generation tests may use a mocked API response containing the same structured fields the real API would receive.

Responsibilities

The Team Leader is responsible for facilitating the testing process, coordinating the availability and schedule of team members acting as testers, and communicating any testing risks to the supervisor. Consistent with the review process described in the Project Proposal, each team member is responsible for testing the components they developed, with the mark-entry and DP-calculation components additionally reviewed by at least one other team member before being considered complete.

Staffing And Training Needs

Testing will be conducted by the project team of six members. Each tester should be familiar with the DP progress calculation rules, threshold bands and trend-classification logic described in the SRS and SDD before executing test cases, so that an unexpected result can be recognised as a defect rather than assumed to be correct.

Schedule

Testing takes place during the Testing and Evaluation phase of the project work plan, after the core mark-upload, DP-calculation and AI-feedback components have been implemented, and before final deployment and project demonstration.

Risks And Contingencies

The Gemini API may be temporarily unavailable, rate-limited, or slow, which could block execution of feedback-related test cases; this is mitigated by mocking the API response for functional tests and reserving live-API calls for a smaller confirmatory subset.

If a component is not complete by its scheduled sprint, dependent test cases may need to be postponed, which could compress the remaining testing schedule.

Team members' limited availability alongside their other academic workload could delay completion of a testing cycle; if this happens, the Team Leader should reprioritise remaining test cases around the features most critical to the Pass/Fail criteria.

Approvals

The Team Leader and the project supervisor, Mrs IN Ezeji, must both agree that testing is complete and that the documented Pass/Fail criteria have been met before the project proceeds to final submission and demonstration.